

# A Continual Learning Survey: Defying Forgetting in Classification Tasks

Matthias De Lange<sup>✉</sup>, Rahaf Aljundi<sup>✉</sup>, Marc Masana<sup>✉</sup>, Sarah Parisot, Xu Jia, Aleš Leonardis, Gregory Slabaugh<sup>✉</sup>, and Tinne Tuytelaars<sup>✉</sup>

**Abstract**—Artificial neural networks thrive in solving the classification problem for a particular rigid task, acquiring knowledge through generalized learning behaviour from a distinct training phase. The resulting network resembles a static entity of knowledge, with endeavours to extend this knowledge without targeting the original task resulting in a catastrophic forgetting. Continual learning shifts this paradigm towards networks that can continually accumulate knowledge over different tasks without the need to retrain from scratch. We focus on task incremental classification, where tasks arrive sequentially and are delineated by clear boundaries. Our main contributions concern: (1) a taxonomy and extensive overview of the state-of-the-art; (2) a novel framework to continually determine the stability-plasticity trade-off of the continual learner; (3) a comprehensive experimental comparison of 11 state-of-the-art continual learning methods; and (4) baselines. We empirically scrutinize method strengths and weaknesses on three benchmarks, considering Tiny Imagenet and large-scale unbalanced iNaturalist and a sequence of recognition datasets. We study the influence of model capacity, weight decay and dropout regularization, and the order in which the tasks are presented, and qualitatively compare methods in terms of required memory, computation time, and storage.

**Index Terms**—Continual learning, lifelong learning, task incremental learning, catastrophic forgetting, classification, neural networks

## 1 INTRODUCTION

IN recent years, machine learning models have been reported to exhibit or even surpass human level performance on individual tasks, such as Atari games [1] or object recognition [2]. While these results are impressive, they are obtained with static models incapable of adapting their behavior over time. As such, this requires restarting the training process each time new data becomes available. In our dynamic world, this practice quickly becomes intractable for data streams or may only be available temporarily due to storage constraints or privacy issues. This calls for systems that adapt continually and keep on learning over time.

Human cognition exemplifies such systems, with a tendency to learn concepts sequentially. Revisiting old concepts by observing examples may occur, but is not essential to preserve this knowledge, and while humans may gradually forget old information, a complete loss of previous knowledge is rarely attested [3].

By contrast, artificial neural networks cannot learn in this manner: they suffer from catastrophic forgetting of old

concepts as new ones are learned [3]. To circumvent this problem, research on artificial neural networks has focused mostly on static tasks, with usually shuffled data to ensure i.i.d. conditions, and vast performance increase by revisiting training data over multiple epochs.

*Continual Learning.* Studies the problem of learning from an infinite stream of data, with the goal of gradually extending acquired knowledge and using it for future learning [4]. Data can stem from changing input domains (e.g. varying imaging conditions) or can be associated with different tasks (e.g. fine-grained classification problems). Continual learning is also referred to as lifelong learning [4], [5], [6], [7], [8], [9], sequential learning [10], [11], [12] or incremental learning [13], [14], [15], [16], [17], [18], [19]. The main criterion is the sequential nature of the learning process, with only a small portion of input data from one or few tasks available at once. The major challenge is to learn without catastrophic forgetting: performance on a previously learned task or domain should not significantly degrade over time as new tasks or domains are added. This is a direct result of a more general problem in neural networks, namely the stability-plasticity dilemma [20], with plasticity referring to the ability of integrating new knowledge, and stability retaining previous knowledge while encoding it. Albeit a challenging problem, progress in continual learning has led to real-world applications starting to emerge [21], [22], [23].

*Scope.* To keep focus, we limit the scope of our study in two ways. First, we only consider the task incremental setting, where data arrives sequentially in batches and one batch corresponds to one task, such as a new set of categories to be learned. In other words, we assume for a given task, all data becomes available simultaneously for offline training. This

- Matthias De Lange, Rahaf Aljundi, and Tinne Tuytelaars are with the Center for Processing Speech and Images, Department Electrical Engineering, 3001 Leuven, Belgium. E-mail: {de.lange.matthias, rahaf.aljundi}@gmail.com, tinne.tuytelaars@esat.kuleuven.be.
- Marc Masana is with the Computer Vision Center, UAB, 08193 Barcelona, Spain. E-mail: marc.masana@ccc.uab.es.
- Sarah Parisot, Xu Jia, Aleš Leonardis, and Gregory Slabaugh are with the Huawei, Montreal, Quebec H3N 1X9, Canada. E-mail: {sarah.parisot, ales.leonardis}@huawei.com, {jiayushenyang, greg.slabaugh}@gmail.com.

Manuscript received 26 Feb. 2020; revised 18 Dec. 2020; accepted 1 Feb. 2021.

Date of publication 5 Feb. 2021; date of current version 3 June 2022.

(Corresponding author: Matthias De Lange.)

Recommended for acceptance by F. Fleuret.

Digital Object Identifier no. 10.1109/TPAMI.2021.3057446

enables learning for multiple epochs over all its training data, repeatedly shuffled to ensure i.i.d. conditions. Importantly, data from previous or future tasks cannot be accessed. Optimizing for a new task in this setting will result in catastrophic forgetting, with significant drops in performance for old tasks, unless special measures are taken. The efficacy of those measures, under different circumstances, is exactly what this paper is about. Furthermore, task incremental learning confines the scope to a multi-head configuration, with an exclusive output layer or head for each task. This is in contrast to the even more challenging class incremental setup with all tasks sharing a single head. This introduces additional interference in learning and increases the number of output nodes to choose from. Instead, we assume it is known which task a given sample belongs to.

Second, we focus on classification problems only, as classification is arguably one of the most established tasks for artificial neural networks, with good performance using relatively simple, standard and well understood network architectures. The setup is described in more detail in Section 2, with Section 7 discussing open issues towards tackling a more general setting.

*Motivation.* There is limited consensus in the literature on experimental setups and datasets to be used. Although papers provide evidence for at least one specific setting of model architecture, combination of tasks and hyperparameters under which the proposed method reduces forgetting and outperforms alternative approaches, there is no comprehensive experimental comparison performed to date.

*Findings.* To establish a fair comparison, the following question needs to be considered: “How can the trade-off between stability and plasticity be set in a consistent manner, using only data from the current task?” For this purpose, we propose a generalizing framework that dynamically determines stability and plasticity for continual learning methods. Within this principled framework, we scrutinize 11 representative approaches in the spectrum of continual learning methods, and find method-specific preferences for configurations regarding model capacity and combinations with typical regularization schemes such as weight decay or dropout. All representative methods generalize well to the nicely balanced small-scale Tiny Imagenet setup. However, this no longer holds when transferring to large-scale real-world datasets, such as the profoundly unbalanced iNaturalist, and especially in the unevenly distributed RecogSeq sequence with highly dissimilar recognition tasks. Nevertheless, rigorous methods based on isolation of parameters seem to withstand generalizing to these challenging settings. Additionally, we find the ordering in which tasks are presented to have insignificant impact on performance. A summary of our main findings can be found in Section 6.8.

*Related Work.* Continual learning has been the subject of several recent surveys [7], [24], [25], [26]. Parisi *et al.* [7] describe a wide range of methods and approaches, yet without an empirical evaluation or comparison. In the same vein, Lesort *et al.* [24] descriptively survey and formalize continual learning, but with an emphasis on dynamic environments for robotics. Pfülb and Gepperth [25] perform an empirical study on catastrophic forgetting and develop a protocol for setting hyperparameters and method evaluation. However, they only consider two methods, namely Elastic Weight

Consolidation (EWC) [27] and Incremental Moment Matching (IMM) [28]. Also, their evaluation is limited to small datasets. Similarly, Kemker *et al.* [29] compare only three methods, EWC [27], PathNet [30] and GeppNet [15]. The experiments are performed on a simple fully connected network, including only 3 datasets: MNIST, CUB-200 and AudioSet. Farquhar and Gal [26] survey continual learning evaluation strategies and highlight shortcomings of the common practice to use one dataset with different pixel permutations (typically permuted MNIST [31]). In addition, they stress that task incremental learning under multi-head settings hides the true difficulty of the problem. While we agree with this statement, we still opted for a multi-head based survey, as it allows comparing existing methods without major modifications. They also propose a couple of desiderata for evaluating continual learning methods. However, their study is limited to the fairly easy MNIST and Fashion-MNIST datasets with far from realistic data encountered in practical applications. None of these works systematically addresses the questions raised above.

*Paper Overview.* First, we describe in Section 2 the widely adopted task incremental setting for this paper. Next, Section 3 surveys different approaches towards continual learning, structuring them into three main groups: replay, regularization-based and parameter isolation methods. An important issue when aiming for a fair method comparison is the selection of hyperparameters, with in particular the learning rate and the stability-plasticity trade-off. Hence, Section 4 introduces a novel framework to tackle this problem without requiring access to data from previous or future tasks. We provide details on the methods selected for our experimental evaluation in Section 5, and describe the actual experiments, main findings, and qualitative comparison in Section 6. We look further ahead in Section 7, highlighting additional challenges in the field, moving beyond the task incremental setting towards true continual learning. Then, we emphasize the relation with other fields in Section 8, and conclude in Section 9. Finally, implementation details and further experimental results are provided in supplemental material, which can be found on the Computer Society Digital Library at <http://doi.ieeecomputersociety.org/10.1109/TPAMI.2021.3057446>. Code is publicly available for reproducibility<sup>1</sup>.

## 2 THE TASK INCREMENTAL LEARNING SETTING

Due to the general difficulty and variety of challenges in continual learning, many methods relax the general setting to an easier task incremental one. The task incremental setting considers a sequence of tasks, receiving training data of just one task at a time to perform training until convergence. Data  $(\mathcal{X}^{(t)}, \mathcal{Y}^{(t)})$  is randomly drawn from distribution  $D^{(t)}$ , with  $\mathcal{X}^{(t)}$  a set of data samples for task  $t$ , and  $\mathcal{Y}^{(t)}$  the corresponding ground truth labels. The goal is to control the statistical risk of all seen tasks given limited or no access to data  $(\mathcal{X}^{(t)}, \mathcal{Y}^{(t)})$  from previous tasks  $t < T$ :

$$\sum_{t=1}^T \mathbb{E}_{(\mathcal{X}^{(t)}, \mathcal{Y}^{(t)})} [\ell(f_t(\mathcal{X}^{(t)}; \theta), \mathcal{Y}^{(t)})], \quad (1)$$

1. Code available at: <https://github.com/MATTDL/CLSURVEY>

with loss function  $\ell$ , parameters  $\theta$ ,  $\mathcal{T}$  the number of tasks seen so far, and  $f_t$  representing the network function for task  $t$ . The definition can be extended to zero-shot learning for  $t > \mathcal{T}$ , but mainly remains future work in the field as discussed in Section 7. For the current task  $\mathcal{T}$ , the statistical risk can be approximated by the empirical risk

$$\frac{1}{N_{\mathcal{T}}} \sum_{i=1}^{N_{\mathcal{T}}} \ell(f(x_i^{(\mathcal{T})}; \theta), y_i^{(\mathcal{T})}) . \quad (2)$$

The main research focus in task incremental learning aims to determine optimal parameters  $\theta^*$  by optimizing (1). However, data is no longer available for old tasks, impeding evaluation of statistical risk for the new parameter values.

The concept ‘task’ refers to an isolated training phase with a new batch of data, belonging to a new group of classes, a new domain, or a different output space. Similar to [32], a finer subdivision into three categories emerges based on the marginal output and input distributions  $P(\mathcal{Y}^{(t)})$  and  $P(\mathcal{X}^{(t)})$  of a task  $t$ , with  $P(\mathcal{X}^{(t)}) \neq P(\mathcal{X}^{(t+1)})$ . First, *class incremental learning* defines an output space for all observed class labels  $\{\mathcal{Y}^{(t)}\} \subset \{\mathcal{Y}^{(t+1)}\}$  with  $P(\mathcal{Y}^{(t)}) \neq P(\mathcal{Y}^{(t+1)})$ . *Task incremental learning* defines  $\{\mathcal{Y}^{(t)}\} \neq \{\mathcal{Y}^{(t+1)}\}$ , which additionally requires task label  $t$  to indicate the isolated output nodes  $\mathcal{Y}^{(t)}$  for current task  $t$ . *Incremental domain learning* defines  $\{\mathcal{Y}^{(t)}\} = \{\mathcal{Y}^{(t+1)}\}$  with  $P(\mathcal{Y}^{(t)}) = P(\mathcal{Y}^{(t+1)})$ . Additionally, *data incremental learning* [33] defines a more general continual learning setting for any data stream without notion of task, class or domain. Our experiments focus on the task incremental learning setting for a sequence of classification tasks.

### 3 CONTINUAL LEARNING APPROACHES

Early works observed the catastrophic interference problem when sequentially learning examples of different input patterns. Several directions have been explored, such as reducing representation overlap [34], [35], [36], [37], [38], replaying samples or virtual samples from the past [8], [39] or introducing dual architectures [40], [41], [42]. However, due to resource restrictions at the time, these works mainly considered few examples (in the order of tens) and were based on specific shallow architectures.

With the recent increased interest in neural networks, continual learning and catastrophic forgetting also received more attention. The influence of dropout [43] and different activation functions on forgetting have been studied empirically for two sequential tasks [31], and several works studied task incremental learning from a theoretical perspective [44], [45], [46], [47].

More recent works have addressed continual learning with longer task sequences and a larger number of examples. In the following, we will review the most important works. We distinguish three families, based on how task specific information is stored and used throughout the sequential learning process:

- Replay methods
- Regularization-based methods
- Parameter isolation methods

Note that our categorization overlaps to some extent with that introduced in previous work [26], [48]. However, we

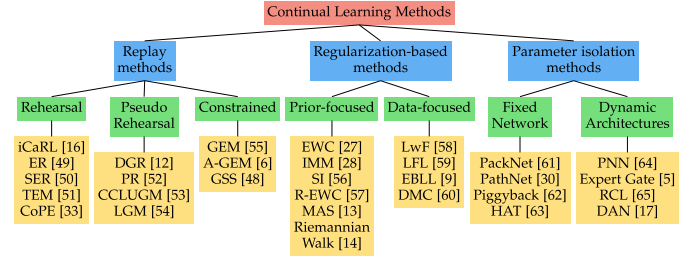


Fig. 1. A tree diagram illustrating the different continual learning families of methods and the different branches within each family. The leaves enlist example methods.

believe it offers a more general overview and covers most existing works. A summary can be found in Fig. 1.

#### 3.1 Replay Methods

This line of work stores samples in raw format or generates pseudo-samples with a generative model. These previous task samples are replayed while learning a new task to alleviate forgetting. They are either reused as model inputs for rehearsal, or to constrain optimization of the new task loss to prevent previous task interference.

*Rehearsal Methods* [16], [49], [50], [51] explicitly retrain on a limited subset of stored samples while training on new tasks. The performance of these methods is upper bounded by joint training on previous and current tasks. Most notable is class incremental learner iCaRL [16], storing a subset of exemplars per class, selected to best approximate class means in the learned feature space. At test time, the class means are calculated for nearest-mean classification based on all exemplars. In data incremental learning, Rolnick *et al.* [49] suggest reservoir sampling to limit the number of stored samples to a fixed budget assuming an i.i.d. data stream. Continual Prototype Evolution (CoPE) [33] combines the nearest-mean classifier approach with an efficient reservoir-based sampling scheme. Additional experiments on rehearsal for class incremental learning are provided in [66].

While rehearsal might be prone to overfitting the subset of stored samples and seems to be bounded by joint training, *constrained optimization* is an alternative solution leaving more leeway for backward/forward transfer. As proposed in GEM [55] under the task incremental setting, the key idea is to only constrain new task updates to not interfere with previous tasks. This is achieved through projecting the estimated gradient direction on the feasible region outlined by previous task gradients through first order Taylor series approximation. A-GEM [6] relaxes the problem to project on one direction estimated by randomly selected samples from a previous task data buffer. Aljundi *et al.* [48] extend this solution to a pure online continual learning setting without task boundaries, proposing to select sample subsets that maximally approximate the feasible region of historical data.

In the absence of previous samples, *pseudo rehearsal* is an alternative strategy used in early works with shallow neural networks. The output of previous model(s) given random inputs are used to approximate previous task samples [39]. With deep networks and large input vectors (e.g., full resolution images), random input cannot cover the input space [52]. Recently, generative models have shown the



ability to generate high quality images [67], [68], opening up possibilities to model the data generating distribution and retrain on generated examples [12]. However, this also adds complexity in training generative models continually, with extra care to balance retrieved examples and avoid mode collapse.

### 3.2 Regularization-Based Methods

This line of works avoids storing raw inputs, prioritizing privacy, and alleviating memory requirements. Instead, an extra regularization term is introduced in the loss function, consolidating previous knowledge when learning on new data. We can further divide these methods into data-focused and prior-focused methods.

#### 3.2.1 Data-Focused Methods

The basic building block in data-focused methods is knowledge distillation from a previous model (trained on a previous task) to the model being trained on the new data. Silver *et al.* [8] first proposed to use previous task model outputs given new task input images, mainly for improving new task performance. It has been re-introduced by LwF [58] to mitigate forgetting and transfer knowledge, using the previous model output as soft labels for previous tasks. Other works [59], [60] have been introduced with related ideas, however, it has been shown that this strategy is vulnerable to domain shift between tasks [5]. In an attempt to overcome this issue, Triki *et al.* [9] facilitate incremental integration of shallow autoencoders to constrain task features in their corresponding learned low dimensional space.

#### 3.2.2 Prior-Focused Methods

To mitigate forgetting, prior-focused methods estimate a distribution over the model parameters, used as prior when learning from new data. Typically, importance of all neural network parameters is estimated, with parameters assumed independent to ensure feasibility. During training of later tasks, changes to important parameters are penalized. Elastic weight consolidation (EWC) [27] was the first to establish this approach. Variational Continual Learning (VCL) introduced a variational framework for this family [69], sprouting a body of Bayesian based works [70], [71]. Zenke *et al.* [56] estimate importance weights online during task training. Aljundi *et al.* [13] suggest unsupervised importance estimation, allowing increased flexibility and online user adaptation as in [21]. Further work extends this to task free settings [72].

### 3.3 Parameter Isolation Methods

This family dedicates different model parameters to each task, to prevent any possible forgetting. When no constraints apply to architecture size, one can grow new branches for new tasks, while freezing previous task parameters [64], [65], or dedicate a model copy to each task [5]. Alternatively, the architecture remains static, with fixed parts allocated to each task. Previous task parts are masked out during new task training, either imposed at parameters level [30], [61], or unit level [63]. These works typically require a task oracle, activating corresponding masks or task branch during prediction. Therefore, they are restrained to a multi-head setup,

incapable to cope with a shared head between tasks. Expert Gate [5] avoids this problem through learning an auto-encoder gate.

## 4 CONTINUAL HYPERPARAMETER FRAMEWORK

Methods tackling the continual learning problem typically involve extra hyperparameters to balance the stability-plasticity trade-off. These hyperparameters are in many cases found via a grid search, using held-out validation data from all tasks. However, this inherently violates the main assumption in continual learning, namely no access to previous task data. This may lead to overoptimistic results, that cannot be reproduced in a true continual learning setting. As it is of our concern in this survey to provide a comprehensive and fair study on the different continual learning methods, we need to establish a principled framework to set hyperparameters without violating the continual learning setting. Besides a fair comparison over the existing approaches, this general strategy dynamically determines the stability-plasticity trade-off, and therefore extends its value to real-world continual learners.

First, our proposed framework assumes only access to new task data to comply with the continual learning paradigm. Next, the framework aims at the best trade-off in stability and plasticity in a dynamic fashion. For this purpose, the set of hyperparameters  $\mathcal{H}$  related to forgetting should be identified for the continual learning method, as we do for all compared methods in Section 5. The hyperparameters are initialized to ensure minimal forgetting of previous tasks. Then, if a predefined threshold on performance is not achieved, hyperparameter values are decayed until reaching the desired performance on the new task validation data. Algorithm 1 illustrates the two main phases of our framework, to be repeated for each new task:

---

#### Algorithm 1. Continual Hyperparameter Selection Framework

---

```

input  $\mathcal{H}$  hyperparameter set,  $\alpha \in [0, 1]$  decaying factor,  $p \in [0, 1]$ 
    accuracy drop margin,  $D^{t+1}$  new task data,  $\Psi$  coarse
    learning rate grid
require  $\theta^t$  previous task model parameters
require  $CLM$  continual learning method
    //Maximal Plasticity Search
1:  $A^* = 0$ 
2: for  $\eta \in \Psi$  do
3:    $A \leftarrow \text{Finetune}(D^{t+1}, \eta; \theta^t) \triangleright$  Finetuning accuracy
4:   if  $A > A^*$  then
5:      $A^*, \eta^* \leftarrow A, \eta \triangleright$  Update best values
    //Stability Decay
6: do
7:    $A \leftarrow CLM(D^{t+1}, \eta^*; \theta^t)$ 
8:   if  $A < (1 - p)A^*$  then
9:      $\mathcal{H} \leftarrow \alpha \cdot \mathcal{H} \triangleright$  Hyperparameter decay
10: while  $A < (1 - p)A^*$ 

```

---

*Maximal Plasticity Search.* First finetunes model copy  $\theta^t$  on new task data, given previous model parameters  $\theta^t$ . The learning rate  $\eta^*$  is obtained via a coarse grid search, which aims for highest accuracy  $A^*$  on a held-out validation set from the new task. The accuracy  $A^*$  represents the best

accuracy that can be achieved while disregarding the previous tasks.

*Stability Decay.* In the second phase we train  $\theta^t$  with the acquired learning rate  $\eta^*$  using the considered continual learning method. Method specific hyperparameters are set to their highest values to ensure minimum forgetting. We define threshold  $p$  indicating tolerated drop in new task performance compared to finetuning  $A^*$ . When this threshold is not met, we decrease hyperparameter values in  $\mathcal{H}$  by scalar multiplication with decay factor  $\alpha$ , and subsequently repeat this phase. This corresponds to increasing model plasticity in order to reach the desired performance threshold. To avoid redundant stability decay iterations, decayed hyperparameters are propagated to later tasks.

## 5 COMPARED METHODS

In Section 6, we carry out a comprehensive comparison between representative methods from each of the three families of continual learning approaches introduced in Section 3. For clarity, we first provide a brief description of the selected methods, highlighting their main characteristics.

### 5.1 Replay Methods

*iCaRL* [16] was the first replay method, focused on learning in a class-incremental way. Assuming fixed allocated memory, it selects and stores samples (exemplars) closest to the feature mean of each class. During training, the estimated loss on new classes is minimized, along with the distillation loss between targets obtained from previous model predictions and current model predictions on the previously learned classes. As the distillation loss strength correlates with preservation of the previous knowledge, this hyperparameter is optimized in our proposed framework. In our study, we consider the task incremental setting, and therefore implement iCaRL also in a multi-head fashion to perform a fair comparison with other methods.

*GEM* [55] exploits exemplars to solve a constrained optimization problem, projecting the current task gradient in a feasible area, outlined by the previous task gradients. The authors observe increased backward transfer by altering the gradient projection with a small constant  $\gamma \geq 0$ , constituting  $\mathcal{H}$  in our framework.

The major drawback of replay methods is limited scalability over the number of classes, requiring additional computation and storage of raw input samples. Although fixing memory limits memory consumption, this also deteriorates the ability of exemplar sets to represent the original distribution. Additionally, storing these raw input samples may also lead to privacy issues.

### 5.2 Regularization-Based Methods

This survey strongly focuses on regularization-based methods, comparing seven methods in this family. The regularization strength correlates to the amount of knowledge retention, and therefore constitutes  $\mathcal{H}$  in our hyperparameter framework.

*Learning Without Forgetting.* (LwF) [58] retains knowledge of preceding tasks by means of knowledge distillation [73]. Before training the new task, network outputs for the new task data are recorded, and are subsequently used during

training to distill prior task knowledge. However, the success of this method depends heavily on the new task data and how strong it is related to prior tasks. Distribution shifts with respect to the previously learned tasks can result in a gradual error build-up to the prior tasks as more dissimilar tasks are added [5], [58]. This error build-up also applies in a class-incremental setup, as shown in [16]. Another drawback resides in the additional overhead to forward all new task data, and storing the outputs. LwF is specifically designed for classification, but has also been applied to other problems, such as object detection [18].

*Encoder Based Lifelong Learning* (EBLL) [9] extends LwF by preserving important low dimensional feature representations of previous tasks. For each task, an under-complete autoencoder is optimized end-to-end, projecting features on a lower dimensional manifold. During training, an additional regularization term impedes the current feature projections to deviate from previous task optimal ones. Although required memory grows linearly with the number of tasks, autoencoder size constitutes only a small fraction of the backbone network. The main computational overhead occurs in autoencoder training, and collecting feature projections for the samples in each optimization step.

*Elastic Weight Consolidation* (EWC) [27] introduces network parameter uncertainty in the Bayesian framework [74]. Following sequential Bayesian estimation, the old posterior of previous tasks  $t < \mathcal{T}$  constitutes the prior for new task  $\mathcal{T}$ , founding a mechanism to propagate old task importance weights. The true posterior is intractable, and is therefore estimated using a Laplace approximation with precision determined by the Fisher Information Matrix (FIM). Near a minimum, this FIM shows equivalence to the positive semi-definite second order derivative of the loss [75], and is in practice typically approximated by the empirical FIM to avoid additional backward passes [76]:

$$\Omega_k^{\mathcal{T}} = \mathbb{E}_{(x,y) \sim D^{\mathcal{T}}} \left[ \left( \frac{\delta \mathcal{L}}{\delta \theta_k^{\mathcal{T}}} \right)^2 \right], \quad (3)$$

with importance weight  $\Omega_k^{\mathcal{T}}$  calculated after training task  $\mathcal{T}$ . Although originally requiring one FIM per task, this can be resolved by propagating a single penalty [77]. Further, the FIM is approximated after optimizing the task, inducing gradients close to zero, and hence very little regularization. This is inherently coped with in our framework, as the regularization strength is initially very high and lowered only to decay stability. Variants of EWC are proposed to address these issues in [78], [57] and [14].

*Synaptic Intelligence* (SI) [56] breaks the EWC paradigm of determining the new task importance weights  $\Omega_k^{\mathcal{T}}$  in a separate phase after training. Instead, they maintain an online estimate  $\omega^{\mathcal{T}}$  during training to eventually attain

$$\Omega_k^{\mathcal{T}} = \sum_{t=1}^{\mathcal{T}} \frac{\omega_k^t}{(\Delta \theta_k^t)^2 + \xi}, \quad (4)$$

with  $\Delta \theta_k^t = \theta_k^t - \theta_k^{t-1}$  the task-specific parameter distance, and damping parameter  $\xi$  avoiding division by zero. Note that the accumulated importance weights  $\Omega_k^{\mathcal{T}}$  are still only updated after training task  $\mathcal{T}$  as in EWC. Further, the knife

cuts both ways for efficient online calculation. First, stochastic gradient descent incurs noise in the approximated gradient during training, and therefore the authors state importance weights tend to be overestimated. Second, catastrophic forgetting in a pretrained network becomes inevitable, as importance weights can't be retrieved. In another work, Riemannian Walk [14] combines the SI path integral with an online version of EWC to measure parameter importance.

*Memory Aware Synapses* (MAS) [13] redefines the parameter importance measure to an unsupervised setting. Instead of calculating gradients of the loss function  $\mathcal{L}$  as in (3), the authors obtain gradients of the squared  $L_2$ -norm of the learned network output function  $f_T$ :

$$\Omega_k^T = \mathbb{E}_{x \sim D^T} \left[ \frac{\delta \|f_T(x; \theta)\|_2^2}{\delta \theta_k^T} \right]. \quad (5)$$

Previously discussed methods require supervised data for the loss-based importance weight estimations, and are therefore confined to available training data. By contrast, MAS enables importance weight estimation on an unsupervised held-out dataset, hence capable of user-specific data adaptation.

*Incremental Moment Matching* (IMM) [28] estimates Gaussian posteriors for task parameters, in the same vein as EWC, but inherently differs in its use of model merging. In the merging step, the mixture of Gaussian posteriors is approximated by a single Gaussian distribution, i.e., a new set of merged parameters  $\theta^{1:T}$  and corresponding covariances  $\Sigma^{1:T}$ . Although the merging strategy implies a single merged model for deployment, it requires storing models during training for each learned task. In their work, two methods for the merge step are proposed: *mean-IMM* and *mode-IMM*. In the former, weights  $\theta_k$  of task-specific networks are averaged following the weighted sum:

$$\theta_k^{1:T} = \sum_t^T \alpha_k^t \theta_k^t, \quad (6)$$

with  $\alpha_k^t$  the mixing ratio of task  $t$ , subject to  $\sum_t^T \alpha_k^t = 1$ . Alternatively, the second merging method *mode-IMM* aims for the mode of the Gaussian mixture, with importance weights obtained from (3):

$$\theta_k^{1:T} = \frac{1}{\Omega_k^T} \sum_t^T \alpha_k^t \Omega_k^t \theta_k^t, \quad \Omega_k^{1:T} = \sum_t^T \alpha_k^t \Omega_k^t. \quad (7)$$

When two models converge to a different local minimum due to independent initialization, simply averaging the models might result in increased loss, as there are no guarantees for a flat or convex loss surface between the two points in parameter space [79]. Therefore, IMM suggests three transfer techniques aiming for an optimal solution in the interpolation of the task-specific models: i) *Weight-Transfer* initializes the new task network with previous task parameters; ii) *Drop-Transfer* is a variant of dropout [80] with previous task parameters as zero point; iii) *L2-transfer* is a variant of L2-regularization, again with previous task parameters redefining the zero point. In this study, we compare *mean-IMM* and *mode-IMM* with both weight-transfer and L2-transfer. As *mean-IMM* is consistently outperformed by *mode-IMM*,

we refer for its full results to appendix available in the online supplemental material.

### 5.3 Parameter Isolation Methods

This family of methods isolates parameters for specific tasks and can guarantee maximal stability by fixing the parameter subsets of previous tasks. Although this refrains stability decay, the capacity used for the new task can be minimized instead to avoid capacity saturation and ensuring stable learning for future tasks.

*PackNet* [61] iteratively assigns parameter subsets to consecutive tasks by constituting binary masks. For this purpose, new tasks establish two training phases. First, the network is trained without altering previous task parameter subsets. Subsequently, a portion of unimportant free parameters are pruned, measured by lowest magnitude. Then, the second training round retrains this remaining subset of important parameters. The pruning mask preserves task performance as it ensures to fix the task parameter subset for future tasks. PackNet allows explicit allocation of network capacity per task, and therefore inherently limits the total number of tasks.  $\mathcal{H}$  contains the per-layer pruning fraction.

*HAT* [63] requires only one training phase, incorporating task-specific embeddings for attention masking. The per-layer embeddings are gated through a Sigmoid to attain unit-based attention masks in the forward pass. The Sigmoid slope is rectified through training of each epoch, initially allowing mask modifications and culminating into near-binary masks. To facilitate capacity for further tasks, a regularization term imposes sparsity on the new task attention mask. Core to this method is constraining parameter updates between two units deemed important for previous tasks, based on the attention masks. Both regularization strength and Sigmoid slope are considered in  $\mathcal{H}$ . We only report results on Tiny Imagenet as difficulties regarding asymmetric capacity and hyperparameter sensitivity prevent using HAT for more difficult setups [81].

We refer to Appendix B.5 and B.6 available in the online supplemental material for a detailed analysis on capacity usage of parameter isolation methods.

## 6 EXPERIMENTS

In this section we first discuss the experimental setup in Section 6.1, followed by a comparison of all the methods on a common BASE model in Section 6.2. The effects of changing the capacity of this model are discussed in Section 6.3. Next, in Section 6.4 we look at the effect of two popular methods for regularization. We continue in Section 6.5, scrutinizing the behaviour of continual learning methods in a real-world setup, abandoning the artificially imposed balance between tasks. In addition, we investigate the effect of the task ordering in both the balanced and unbalanced setup in Section 6.6, and elucidate a qualitative comparison in Table 9 of Section 6.7. Finally, Section 6.8 summarizes our main findings in Table 10.

### 6.1 Experimental Setup

*Datasets.* We conduct image classification experiments on three datasets, the main characteristics of which are



TABLE 1  
The Balanced Tiny Imagenet, and Unbalanced iNaturalist and RecogSeq Dataset Characteristics

|                 | Tiny Imagenet | iNaturalist   | RecogSeq    |
|-----------------|---------------|---------------|-------------|
| Tasks           | 10            | 10            | 8           |
| Classes/task    | 20            | 5 to 314      | 10 to 200   |
| Train data/task | 8k            | 0.6k to 66k   | 2k to 73k   |
| Val. data/task  | 1k            | 0.1k to 9k    | 0.6k to 13k |
| Task selection  | random class  | supercategory | dataset     |

summarized in Table 1. First, we use the *Tiny Imagenet* dataset [82]. This is a subset of 200 classes from ImageNet [83], rescaled to image size  $64 \times 64$ . Each class contains 500 samples subdivided into training (80 percent) and validation (20 percent), and 50 samples for evaluation. In order to construct a balanced dataset, we assign an equal amount of 20 randomly chosen classes to each task in a sequence of 10 consecutive tasks. This task incremental setting allows using an oracle at test time for our evaluation per task, ensuring all tasks are roughly similar in terms of difficulty, size, and distribution, making the interpretation of the results easier.

The second dataset is based on *iNaturalist* [84], which aims for a more real-world setting with a large number of fine-grained categories and highly imbalanced classes. On top, we impose task imbalance and domain shifts between tasks by assigning 10 super-categories of species as separate tasks. We selected the most balanced 10 super-categories from the total of 14 and only retained categories with at least 100 samples. More details on the statistics for each of these tasks can be found in Table 8. We only utilize the training data, subdivided in training (70 percent), validation (20 percent) and evaluation (10 percent) sets, with all images measuring  $800 \times 600$ .

Third, we adopt a sequence of 8 highly diverse recognition tasks (*RecogSeq*) as in [5] and [13], which also targets distributing an imbalanced number of classes, and reaches beyond object recognition with scene and action recognition. This sequence is composed of 8 consecutive datasets, from fine-grained to coarse classification of objects, actions and scenes, going from flowers, scenes, birds and cars, to aircrafts, actions, letters and digits. Details are provided in Table 2, with validation and evaluation split following [5].

In this survey we scrutinize the effects of different task orderings for both Tiny Imagenet and iNaturalist in Section 6.6. Apart from that section, discussed results on both datasets are performed on a random ordering of tasks, and the fixed dataset ordering for RecogSeq.

TABLE 2  
The RecogSeq Dataset Sequence Details

| Task                           | Classes | Samples      |            |             |
|--------------------------------|---------|--------------|------------|-------------|
|                                |         | <i>Train</i> | <i>Val</i> | <i>Test</i> |
| <b>Oxford Flowers</b> [86]     | 102     | 2040         | 3074       | 3075        |
| <b>MIT Scenes</b> [87]         | 67      | 5360         | 670        | 670         |
| <b>Caltech-UCSD Birds</b> [88] | 200     | 5994         | 2897       | 2897        |
| <b>Stanford Cars</b> [89]      | 196     | 8144         | 4020       | 4021        |
| <b>FGVC-Aircraft</b> [90]      | 100     | 6666         | 1666       | 1667        |
| <b>VOC Actions</b> [85]        | 11      | 3102         | 1554       | 1554        |
| <b>Letters</b> [91]            | 52      | 6850         | 580        | 570         |
| <b>SVHN</b> [92]               | 10      | 73257        | 13016      | 13016       |

Note that VOC Actions represents the human action classification subset of the VOC challenge 2012 [85].

*Models.* We summarize in Table 3 the models used for experiments in this work. Due to the limited size of *Tiny Imagenet* we can easily run experiments with different models. This allows to analyze the influence of model capacity (Section 6.3) and regularization for each model configuration (Section 6.4). This is important, as the effect of model size and architecture on the performance of different continual learning methods has not received much attention so far. We configure a **BASE** model, two models with less (**SMALL**) and more (**WIDE**) units per layer, and a **DEEP** model with more layers. The models are based on a VGG configuration [93], but with less parameters due to the small image size. We reduce the feature extractor to comprise 4 max-pooling layers, each preceded by a stack of identical convolutional layers with a consistent  $3 \times 3$  receptive field. The first max-pooling layer is preceded by one conv. layer with 64 filters. Depending on the model, we increase subsequent conv. layer stacks with a multiple of factor 2. The models have a classifier consisting of 2 fully connected layers with each 128 units for the **SMALL** model and 512 units for the other three models. The multi-head setting imposes a separate fully connected layer with softmax for each task, with the number of outputs defined by the classes in that task. The detailed description can be found in Appendix A available in the online supplemental material.

The size of *iNaturalist* and *RecogSeq* impose arduous learning. Therefore, we conduct the experiments solely for AlexNet [94], pretrained on ImageNet.

*Evaluation Metrics.* To measure performance in the continual learning setup, we evaluate accuracy and forgetting per task, after training each task. We define this measure of forgetting [14] as the difference between the expectation of acquired knowledge of a task, i.e., the accuracy when first learning a task, and the accuracy obtained after training one

TABLE 3  
Models Used for Tiny Imagenet, iNaturalist, and RecogSeq Experiments

|                              | Model        | Feature Extractor |               |                | Classifier (w/o head) |                  | Total Parameters  | Pretrained | Multi-head    |
|------------------------------|--------------|-------------------|---------------|----------------|-----------------------|------------------|-------------------|------------|---------------|
|                              |              | <i>Conv.</i>      | <i>Layers</i> | <i>MaxPool</i> | <i>Parameters</i>     | <i>FC layers</i> | <i>Parameters</i> |            |               |
| <b>Tiny Imagenet</b>         | <b>SMALL</b> | 6                 | 4             | 334k           | 2                     | 279k             | 613k              | ×          | ✓             |
|                              | <b>BASE</b>  | 6                 | 4             | 1.15m          | 2                     | 2.36k            | 3.51m             | ×          | ✓             |
|                              | <b>WIDE</b>  | 6                 | 4             | 4.5m           | 2                     | 4.46m            | 8.95m             | ×          | ✓             |
|                              | <b>DEEP</b>  | 20                | 4             | 4.28m          | 2                     | 2.36k            | 6.64m             | ×          | ✓             |
| <b>iNaturalist/ RecogSeq</b> |              | AlexNet           | 5             | 3              | 2.47m                 | 2 (with Dropout) | 54.5m             | 57.0m      | ✓(Imagenet) ✓ |

or more additional tasks. In the figures, we focus on evolution of accuracy for each task as more tasks are added. In the tables, we report average accuracy and average forgetting on the final model, obtained by evaluating each task after learning the entire task sequence.

*Baselines.* The discussed continual learning methods in Section 5 are compared against several baselines:

- 1) *Finetuning* starts from the previous task model to optimize current task parameters. This baseline greedily trains each task without considering previous task performance, hence introducing catastrophic forgetting, and representing the minimum desired performance.
- 2) *Joint training* considers all data in the task sequence simultaneously, hence violating the continual learning setup (indicated with appended '\*' in reported results). This baseline provides a target reference performance.

For the replay methods we consider two additional finetuning baselines, extending baseline (1) with the benefit of using exemplars:

- 3) *Basic rehearsal with Full Memory (R-FM)* fully exploits total available exemplar memory  $R$ , incrementally dividing equal capacity over all previous tasks. This is a baseline for replay methods defining memory management policies to exploit all memory (e.g., iCaRL).
- 4) *Basic rehearsal with Partial Memory (R-PM)* pre-allocates fixed exemplar memory  $R/T$  over all tasks, assuming the amount of tasks  $T$  is known beforehand. This is used by methods lacking memory management policies (e.g. GEM).

*Replay Buffers.* Replay methods (GEM, iCaRL) and corresponding baselines (R-PM, R-FM) can be configured with arbitrary replay buffer size. Too large buffers would result in unfair comparison to regularization and parameter isolation methods, with in its limit even holding all data for the previous tasks, as in joint learning, which is not compliant with the continual learning setup. Therefore, we use the memory required to store the BASE model as a basic reference for the amount of exemplars. This corresponds to the additional memory needed to propagate importance weights in the prior-focused methods (EWC, SI, MAS). For Tiny Imagenet, this gives a total replay buffer capacity of 4.5k exemplars. We also experiment with a buffer of 9k exemplars to examine the influence of increased buffer capacity. Note that the BASE reference implies replay methods to use a significantly different amount of memory than the non-replay based methods; more memory for the SMALL model), and less memory for the WIDE and DEEP models. Comparisons between replay and non-replay based methods should thus only be done for the BASE model. In the following, notation of the methods without replay buffer size refers to the default 4.5k buffer.

*Learning Details.* During training the models are optimized with stochastic gradient descent with a momentum of 0.9. Training lasts for 70 epochs unless preempted by early stopping. Standard AlexNet is configured with dropout for iNaturalist and RecogSeq. For Tiny Imagenet we do not use any form of regularization by default, except for

Section 6.4 explicitly scrutinizing regularization influence. The framework we proposed in Section 4 can only be exploited in the case of forgetting-related hyperparameters, in which case we set  $p = 0.2$  and  $\alpha = 0.5$ . All methods discussed in Section 5 satisfy this requirement, except for IMM with L2 transfer for which we could not identify a specific hyperparameter related to forgetting. For specific implementation details about the continual learning methods we refer to Appendix A available in the online supplemental material.

## 6.2 Comparing Methods on the BASE Network

*Tiny Imagenet.* We start evaluation of continual learning methods discussed in Section 5 with a comparison using the BASE network, on the Tiny Imagenet dataset, with random ordering of tasks. This provides a balanced setup, making interpretation of results easier.

Fig. 2 shows results for all three families. Each of these figures consists of 10 subpanels, with each subpanel showing test accuracy evolution for a specific task (e.g. Task 1 for the leftmost panel) as more tasks are added for training. Since the  $n$ -th task is added for training only after  $n$  steps, curves get shorter as we move to subpanels on the right.

### 6.2.1 Discussion

*General Observations.* As a reference, it is relevant to highlight the soft upper bound obtained when training all tasks jointly. This is indicated by the star symbol in each of the subpanels. For the Tiny Imagenet dataset, all tasks have the same number of classes, same amount of training data and similar level of difficulty, resulting in similar accuracies for all tasks under the joint training scheme. The average accuracy for joint training on this dataset is 55.70 percent, while random guessing would result in 5 percent.

Further, as reported ample times in literature, the finetuning baseline suffers severely from catastrophic forgetting: initially good results are obtained when learning a task, but as soon as a new task is added, performance drops, resulting in poor average accuracy of only 21.30 and 26.90 percent average forgetting.

With 49.13 percent average accuracy, PackNet shows highest overall performance after training all tasks. When learning new tasks, due to compression PackNet only has a fraction of the total model capacity available. Therefore, it typically performs worse on new tasks compared to other methods. However, by fixing task parameters through masking, it allows complete knowledge retention until the end of the task sequence (no forgetting yields flat curves in the figure), resulting in highest accumulation of knowledge. This holds at least when working with long sequences, where forgetting errors gradually build up for other methods.

MAS and iCaRL show competitive results w.r.t. PackNet (resp. 46.90 and 47.27 percent). iCaRL starts with a significantly lower accuracy for new tasks, due to its nearest-neighbour based classification, but improves over time, especially for the first few tasks. Further, doubling replay-buffer size to 9k enhances iCaRL performance to 48.76 percent.

*Regularization-Based Methods.* In prior experiments where we did a grid search over a range of hyperparameters over the whole sequence (rather than using the framework



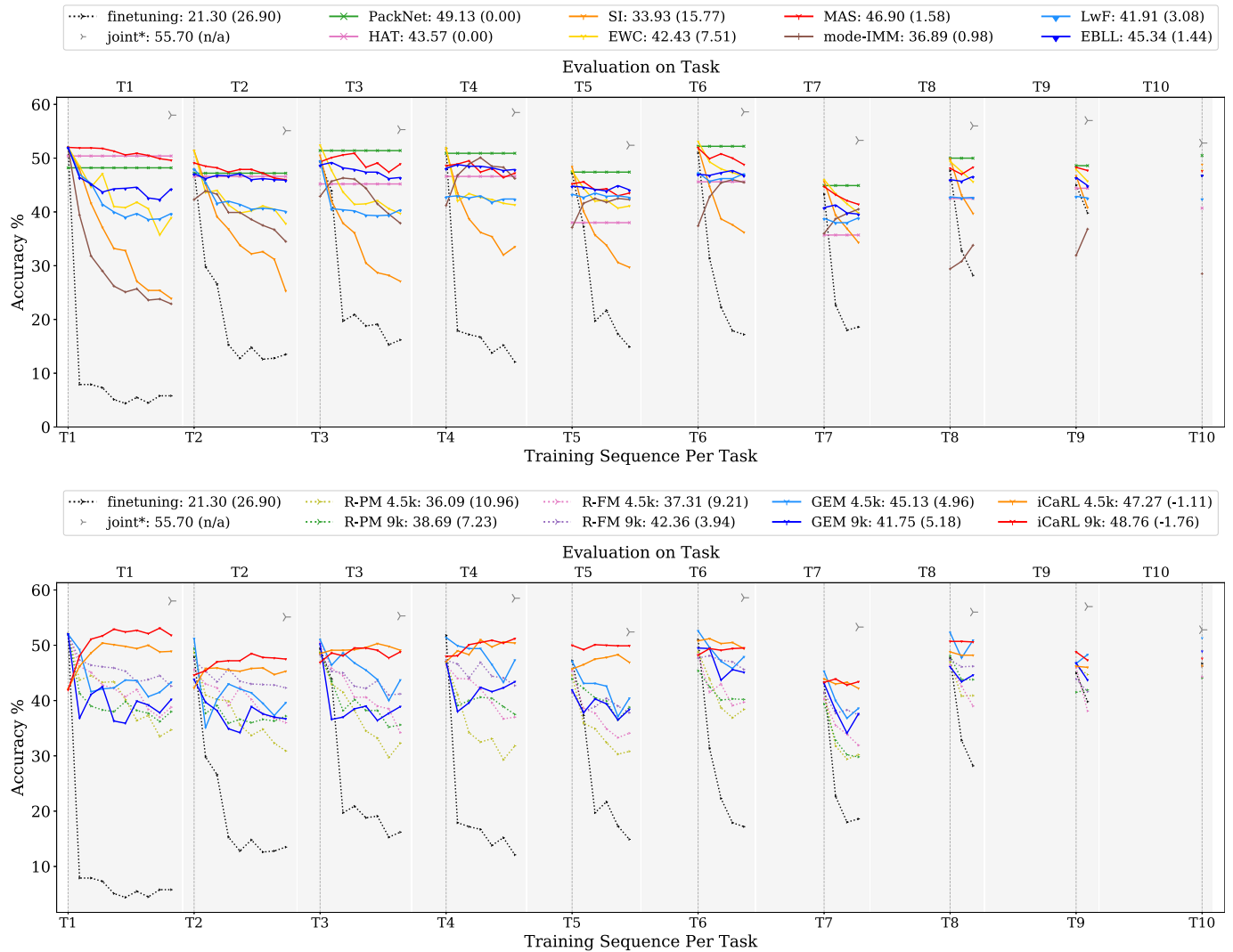


Fig. 2. Parameter isolation and regularization-based methods (top) and replay methods (bottom) on Tiny Imagenet for the BASE model with random ordering, reporting average accuracy (forgetting) in the legend.

introduced in Section 4), we observed MAS to be remarkably more robust to the choice of hyperparameter values compared to the two related methods EWC and SI. Switching to the continual hyperparameter selection framework described in Section 4, this robustness leads to superior results for MAS with 46.90 percent compared to the other two (42.43 and 33.93 percent). Especially improved forgetting stands out (1.58 versus 7.51 and 15.77 percent). Further, SI underperforms compared to EWC on Tiny Imagenet. We hypothesize this may be due to overfitting we observed when training on the BASE model (see results Appendix B.2 available in the online supplemental material). SI inherently constrains parameter importance estimation to the training data only, which is opposed to EWC and MAS able to determine parameter importance both on validation and training data in a separate phase after task training. Further, mode-IMM catastrophically forgets in evaluation on the first task, but shows transitory recovering through backward transfer in all subsequent tasks. Overall for Tiny Imagenet, IMM does not seem competitive with the other strategies for continual learning - especially if one takes into account that they need to store all previous task models, making them much more expensive in terms of storage.

The data-driven methods LwF and EBLL obtain similar results as EWC (41.91 and 45.34 versus 42.43 percent). EBLL improves over LwF by residing closer to the optimal task representations, lowering average forgetting and improving accuracy. Apart from the first few tasks, curves for these methods are quite flat, indicating low levels of forgetting.

**Replay Methods.** iCaRL starts from the same model as GEM and the regularization-based methods, but uses a feature classifier based on a nearest-neighbor scheme. As indicated earlier, this results in lower accuracies on the first task after training. Remarkably, for about half of the tasks, the iCaRL accuracy increases when learning additional tasks, resulting in a salient negative average forgetting of -1.11 percent. Such level of backward transfer is unusual. After training all ten tasks, a competitive result of 47.27 percent average accuracy can be reported. Comparing iCaRL to its baseline R-FM shows significant improvements over basic rehearsal (47.27 versus 37.31 percent). Doubling the size of the replay buffer (iCaRL 9k) increases performance even more, pushing iCaRL closer to PackNet with 48.76 percent average accuracy.

The results of GEM are significantly lower than those of iCaRL (45.13 versus 47.27 percent). GEM is originally

TABLE 4

*Parameter Isolation and Regularization-Based Methods results on Tiny Imagenet for Different Models With Random (Top), Easy to Hard (Middle) and Hard to Easy (Bottom) Ordering of Tasks, Reporting Average Accuracy (Forgetting)*

| Model | finetuning    | joint*      | PackNet      | HAT           | SI            | EWC           | MAS           | LwF           | EBLL          | mode-IMM      |
|-------|---------------|-------------|--------------|---------------|---------------|---------------|---------------|---------------|---------------|---------------|
| SMALL | 16.25 (34.84) | 57.00 (n/a) | 46.68 (0.00) | 44.19 (0.00)  | 23.91 (23.26) | 45.13 (0.86)  | 40.58 (0.78)  | 44.06 (-0.44) | 44.13 (-0.53) | 29.63 (3.06)  |
| BASE  | 21.30 (26.90) | 55.70 (n/a) | 49.13 (0.00) | 43.57 (0.00)  | 33.93 (15.77) | 42.43 (7.51)  | 46.90 (1.58)  | 41.91 (3.08)  | 45.34 (1.44)  | 36.89 (0.98)  |
| WIDE  | 25.28 (24.15) | 57.29 (n/a) | 47.64 (0.00) | 43.78 (0.50)  | 33.86 (15.16) | 31.10 (17.07) | 45.08 (2.58)  | 46.79 (1.19)  | 46.25 (1.72)  | 36.42 (1.66)  |
| DEEP  | 20.82 (20.60) | 51.04 (n/a) | 35.54 (0.00) | 8.06 (3.21)   | 24.53 (12.15) | 29.14 (7.92)  | 33.58 (0.91)  | 32.28 (2.58)  | 27.78 (3.14)  | 27.51 (0.47)  |
| Model | finetuning    | joint*      | PackNet      | HAT           | SI            | EWC           | MAS           | LwF           | EBLL          | mode-IMM      |
| SMALL | 16.06 (35.40) | 57.00 (n/a) | 49.21 (0.00) | 43.89 (0.00)  | 35.98 (13.02) | 40.18 (7.88)  | 44.29 (1.77)  | 45.04 (1.89)  | 42.07 (1.73)  | 26.13 (3.03)  |
| BASE  | 23.26 (24.85) | 55.70 (n/a) | 50.57 (0.00) | 43.93 (-0.02) | 33.36 (14.28) | 34.09 (13.25) | 44.02 (1.30)  | 43.46 (2.53)  | 43.60 (1.71)  | 36.81 (-1.17) |
| WIDE  | 19.67 (29.95) | 57.29 (n/a) | 47.37 (0.00) | 41.63 (0.25)  | 34.84 (13.14) | 28.35 (21.16) | 45.58 (1.52)  | 42.66 (1.07)  | 44.12 (3.46)  | 38.68 (-1.09) |
| DEEP  | 23.90 (17.91) | 51.04 (n/a) | 34.56 (0.00) | 28.64 (0.87)  | 24.56 (14.41) | 24.45 (15.22) | 35.23 (2.37)  | 27.61 (3.70)  | 29.73 (1.95)  | 25.89 (-2.09) |
| Model | finetuning    | joint*      | PackNet      | HAT           | SI            | EWC           | MAS           | LwF           | EBLL          | mode-IMM      |
| SMALL | 18.62 (28.68) | 57.00 (n/a) | 44.42 (0.00) | 33.52 (1.55)  | 40.39 (4.50)  | 41.62 (3.54)  | 40.90 (1.35)  | 42.36 (0.63)  | 43.60 (-0.05) | 24.95 (1.66)  |
| BASE  | 21.17 (22.73) | 55.70 (n/a) | 43.17 (0.00) | 40.21 (0.18)  | 40.79 (2.58)  | 41.83 (1.51)  | 41.98 (0.14)  | 41.58 (1.40)  | 41.57 (0.82)  | 34.58 (0.23)  |
| WIDE  | 25.25 (22.69) | 57.29 (n/a) | 45.53 (0.00) | 41.76 (0.07)  | 37.91 (8.05)  | 29.91 (15.77) | 43.55 (-0.29) | 43.87 (1.26)  | 42.42 (0.85)  | 35.24 (-0.82) |
| DEEP  | 15.33 (20.66) | 51.04 (n/a) | 30.73 (0.00) | 27.21 (3.21)  | 22.97 (13.23) | 22.32 (13.86) | 32.99 (2.19)  | 30.77 (2.51)  | 30.15 (2.67)  | 26.38 (1.20)  |

designed for an online learning setup, while in this comparison each method can exploit multiple epochs for a task. Additional experiments in Appendix B.3 available in the online supplemental material compare GEM sensitivity to the amount of epochs with iCaRL, from which we procure a GEM setup with 5 epochs for all the experiments in this work. Furthermore, the lack of memory management policy in GEM gives iCaRL a compelling advantage w.r.t. the amount of exemplars for the first tasks, e.g., training Task 2 comprises a replay buffer of Task 1 with factor 10 (number of tasks) more exemplars. Surprisingly, GEM 9k with twice as much exemplar capacity doesn't perform better than GEM 4.5k. This unexpected behavior may be due to the random exemplar selection yielding a less representative subset. Nevertheless, GEM convincingly improves accuracy of the basic replay baseline R-PM with the same partial memory scheme (45.13 versus 36.09 percent).

Comparing the two basic rehearsal baselines that use the memory in different ways, we observe the scheme exploiting full memory from the start (R-FM) giving significantly better results than R-PM for tasks 1 to 7, but not for the last three. As more tasks are seen, both baselines converge to the same memory scheme, where for the final task R-FM allocates an equal portion of memory to each task in the sequence, hence equivalent to R-PM.

*Parameter Isolation Methods.* Both HAT and PackNet exhibit the virtue of masking, reporting zero forgetting (see the two flat curves). HAT performs better for the first task, but builds up a salient discrepancy with PackNet for later tasks. PackNet rigorously assigns layer-wise portions of network capacity per task, while HAT imposes sparsity by regularization, which does not guarantee uniform capacity over layers and tasks.

### 6.3 Effects of Model Capacity

*Tiny Imagenet.* A crucial design choice in continual learning concerns network capacity. Aiming to learn a long task sequence, a high capacity model seems preferable. However, learning the first task using such model, with only data from a single task, holds the risk of overfitting, jeopardizing generalization performance. So far, we compared all methods on the BASE model. Next, we study effects of

extending or reducing capacity in this architecture. Details of the models are given in Section 6.1 and Appendix A available in the online supplemental material. In the following, we discuss results of model choice, again using the random ordering of Tiny Imagenet. These results are summarized in the top part of Table 4 for parameter isolation and regularization-based methods and Table 5 for replay methods<sup>2</sup>.

#### 6.3.1 Discussion

*Overfitting.* We observed overfitting for several models and methods, and not only for the WIDE and DEEP models. Comparing different methods, SI seems quite vulnerable to overfitting issues, while PackNet prevents overfitting to some extent, thanks to the network compression phase.

*General Observations.* Selecting highest accuracy disregarding which model is used, iCaRL 9k and PackNet remain on the lead with 49.94 and 49.13 percent. Further, LwF shows to be competitive with MAS (resp. 46.79 and 46.90 percent).

*Baselines.* Taking a closer look at the finetuning baseline results (see purple box in Table 4), we observe that it does not reach the same level of accuracy with the SMALL model as with the other models. In particular, the initial accuracies are similar, yet the level of forgetting is much more severe, due to the limited model capacity to learn new tasks.

Joint training results on the DEEP network (blue box) are inferior compared to shallower networks, implying the deep architecture to be less suited to accumulate all knowledge from the task sequence. As performance of joint learning serves as a soft upper bound for the continual learning methods, this already serves as an indication that a deep model may not be optimal.

In accordance, replay baselines R-PM and R-FM show to be quite model agnostic with all but the DEEP model performing very similar. The baselines experience both less forgetting and increased average accuracy when doubling replay buffer size.

2. Note that, for all of our observations, we also checked the results obtained for the two other task orders (reported in the middle and bottom part of Tables 4 and 5). The reader can check that the observations are quite consistent.

TABLE 5  
*Replay Methods Results on Tiny Imagenet for Different Models with Random (top), Easy to Hard (Middle) and Hard to Easy (Bottom) Ordering of Tasks, Reporting Average Accuracy (Forgetting)*

| Model | finetuning    | joint*      | R-PM 4.5k     | R-PM 9k      | R-FM 4.5k    | R-FM 9k      | GEM 4.5k      | GEM 9k       | iCaRL 4.5k    | iCaRL 9k      |
|-------|---------------|-------------|---------------|--------------|--------------|--------------|---------------|--------------|---------------|---------------|
| SMALL | 16.25 (34.84) | 57.00 (n/a) | 36.97 (12.21) | 40.11 (9.23) | 39.60 (9.57) | 41.35 (6.33) | 39.44 (7.38)  | 41.59 (4.17) | 43.22 (-1.39) | 46.32 (-1.07) |
| BASE  | 21.30 (26.90) | 55.70 (n/a) | 36.09 (10.96) | 38.69 (7.23) | 37.31 (9.21) | 42.36 (3.94) | 45.13 (4.96)  | 41.75 (5.18) | 47.27 (-1.11) | 48.76 (-1.76) |
| WIDE  | 25.28 (24.15) | 57.29 (n/a) | 36.47 (12.45) | 41.51 (5.15) | 39.25 (9.26) | 41.53 (6.02) | 40.32 (6.97)  | 44.23 (3.94) | 44.20 (-1.43) | 49.94 (-2.80) |
| DEEP  | 20.82 (20.60) | 51.04 (n/a) | 27.61 (7.14)  | 28.99 (6.27) | 32.26 (3.33) | 33.10 (4.70) | 29.66 (6.57)  | 23.75 (6.93) | 36.12 (-0.93) | 37.16 (-1.64) |
| Model | finetuning    | joint*      | R-PM 4.5k     | R-PM 9k      | R-FM 4.5k    | R-FM 9k      | GEM 4.5k      | GEM 9k       | iCaRL 4.5k    | iCaRL 9k      |
| SMALL | 16.06 (35.40) | 57.00 (n/a) | 37.09 (11.64) | 40.82 (8.08) | 38.47 (9.62) | 40.81 (7.73) | 39.07 (9.45)  | 42.16 (6.93) | 48.55 (-1.17) | 46.67 (-2.03) |
| BASE  | 23.26 (24.85) | 55.70 (n/a) | 36.70 (8.88)  | 39.85 (6.59) | 38.27 (8.35) | 40.95 (6.21) | 30.73 (10.39) | 40.20 (7.41) | 46.30 (-1.53) | 47.47 (-2.22) |
| WIDE  | 19.67 (29.95) | 57.29 (n/a) | 39.43 (9.10)  | 42.55 (5.32) | 40.67 (7.52) | 43.23 (4.09) | 44.49 (4.66)  | 40.78 (8.14) | 48.13 (-2.01) | 44.59 (-2.44) |
| DEEP  | 23.90 (17.91) | 51.04 (n/a) | 30.98 (6.32)  | 32.06 (4.16) | 29.62 (6.85) | 33.92 (3.56) | 25.29 (12.79) | 29.09 (6.30) | 32.46 (-0.35) | 34.64 (-1.25) |
| Model | finetuning    | joint*      | R-PM 4.5k     | R-PM 9k      | R-FM 4.5k    | R-FM 9k      | GEM 4.5k      | GEM 9k       | iCaRL 4.5k    | iCaRL 9k      |
| SMALL | 18.62 (28.68) | 57.00 (n/a) | 33.69 (11.06) | 37.68 (6.34) | 37.18 (7.13) | 38.47 (4.88) | 38.87 (6.78)  | 39.28 (7.44) | 46.81 (-0.92) | 46.90 (-1.61) |
| BASE  | 21.17 (22.73) | 55.70 (n/a) | 32.90 (8.98)  | 34.34 (7.50) | 33.53 (9.08) | 36.83 (5.43) | 35.11 (7.44)  | 35.95 (6.68) | 43.29 (-0.47) | 44.52 (-1.71) |
| WIDE  | 25.25 (22.69) | 57.29 (n/a) | 34.85 (7.94)  | 40.28 (6.67) | 36.70 (6.63) | 37.46 (5.62) | 37.27 (7.86)  | 37.91 (6.94) | 41.49 (-1.49) | 49.66 (-2.90) |
| DEEP  | 15.33 (20.66) | 51.04 (n/a) | 24.22 (6.67)  | 23.42 (5.93) | 25.81 (6.06) | 29.99 (2.91) | 27.08 (6.47)  | 31.28 (5.52) | 30.95 (0.85)  | 37.93 (-1.35) |

**SMALL Model.** Finetuning and SI suffer from severe forgetting ( $> 20$  percent) imposed by the decreased capacity of the network (see red underlining), making these combinations worthless in practice. Other methods experience alleviated forgetting, with EWC most saliently benefitting from a small network (see green underlinings).

**WIDE Model.** Remarkable in WIDE model results is SI consistently outperforming EWC, in contrast with other model results (see orange box; even more clear in middle and bottom tables in Table 4, which will be discussed later). EWC performs worse for the high capacity WIDE and DEEP models and, as previously discussed, attains best performance for the SMALL model. LwF, EBL and PackNet mainly reach their top performance when using the WIDE model, with SI performing most stable on both the WIDE and BASE models. IMM also shows increased performance when using the BASE and WIDE model.

**DEEP Model.** Over the whole line of methods (yellow box), extending the BASE model with additional convolutional layers results in lower performance. As we already observed overfitting on the BASE model, additional layers may introduce extra unnecessary layers of abstraction. For the DEEP model iCaRL outperforms all continual learning methods with both memory sizes. HAT deteriorates significantly for the DEEP model, especially in the random ordering, for which we observe issues regarding asymmetric allocation of network capacity. This makes the method sensitive to ordering of tasks, as early fixed parameters determine performance for later tasks at saturated capacity. We discuss this in more detail in Appendix B.5 available in the online supplemental material.

**Model Agnostic.** Over all orderings which will be discussed in Section 6.6, some methods don't exhibit a preference for any model, except for the common aversion for the detrimental DEEP model. This is in general most salient for all replay methods in Table 5, but also for PackNet, MAS, LwF and EBL in Table 4.

**Conclusion.** We can conclude that (too) deep model architectures do not provide a good match with the continual learning setup. For the same amount of feature extractor parameters, WIDE models obtain significant better results (on average 11 percent better over all methods in Table 4 and Table 5). Also too small models should be avoided, as the

limited available capacity can cause forgetting. On average we observe a modest 0.89 percent more forgetting for the SMALL model compared to the BASE model. At the same time, for some models, poor results may be due to overfitting, which can possibly be overcome using regularization, as we will study next.

## 6.4 Effects of Regularization

**Tiny Imagenet.** In the previous subsection we mentioned the problem of overfitting in continual learning. Although an evident solution would be to apply regularization, this might interfere with the continual learning methods. Therefore, we investigate the effects of two popular regularization methods, namely dropout and weight decay, for the parameter isolation and regularization-based methods in Table 6, and for the replay methods in Table 7. For dropout we set the probability of retaining the units to  $p = 0.5$ , and weight decay applies a regularization strength of  $\lambda = 10^{-4}$ . Any form of regularization is applied in both phases of our framework.

### 6.4.1 Discussion

**General Observations.** In Tables 6 and 7, negative results (with regularization hurting performance) are underlined in red. This occurs repeatedly, especially in combination with the SMALL model or with weight decay. Over all methods and models dropout mainly shows to be fruitful. This is consistent with earlier observations [31]. There are, however, a few salient exceptions (discussed below). Weight decay over the whole line mainly improves the wide network accuracies. In the following, we will describe the most notable observations and exceptions to the main tendencies.

**Finetuning.** Goodfellow *et al.* observe reduced catastrophic forgetting in a transfer learning setup with finetuning when using dropout [31]. Extended to learning a sequence of 10 consecutive tasks in our experiments, finetuning consistently benefits from dropout regularization. This is opposed to weight decay, resulting in increased forgetting and a lower performance on the final model. In spite of good results for dropout, we regularly observe an increase in the level of forgetting, which is compensated for by starting from a better initial model, due to a reduction in



TABLE 6  
Parameter Isolation and Regularization-Based Methods: Dropout and Weight Decay for Tiny Imagenet Models

| Model |              | finetuning    | joint*      | PackNet      | HAT          | SI            | EWC           | MAS           | LwF           | EBLL          | mode-IMM      |
|-------|--------------|---------------|-------------|--------------|--------------|---------------|---------------|---------------|---------------|---------------|---------------|
| SMALL | Plain        | 16.25 (34.84) | 57.00 (n/a) | 49.09 (0.00) | 44.19 (0.00) | 23.91 (23.26) | 45.13 (0.86)  | 40.58 (0.78)  | 44.06 (-0.44) | 44.13 (-0.53) | 29.63 (3.06)  |
|       | Dropout      | 19.52 (32.62) | 55.97 (n/a) | 50.73 (0.00) | 26.87 (2.46) | 38.34 (9.24)  | 40.02 (7.57)  | 40.26 (7.63)  | 31.56 (18.38) | 34.14 (13.20) | 29.35 (0.90)  |
|       | Weight Decay | 15.06 (34.61) | 56.96 (n/a) | 49.90 (0.00) | 42.44 (0.00) | 37.99 (6.43)  | 41.22 (1.52)  | 37.37 (4.43)  | 41.66 (-0.28) | 42.67 (-0.63) | 30.59 (0.93)  |
| BASE  | Plain        | 21.30 (26.90) | 55.70 (n/a) | 47.67 (0.00) | 43.57 (0.00) | 33.93 (15.77) | 42.43 (7.51)  | 46.90 (1.58)  | 41.91 (3.08)  | 45.34 (1.44)  | 36.89 (0.98)  |
|       | Dropout      | 29.23 (26.44) | 61.43 (n/a) | 54.28 (0.00) | 38.11 (0.36) | 43.15 (10.83) | 42.09 (12.54) | 48.98 (0.87)  | 41.49 (8.72)  | 44.66 (7.66)  | 34.20 (1.44)  |
|       | Weight Decay | 19.14 (29.31) | 57.12 (n/a) | 48.28 (0.00) | 44.97 (0.19) | 39.65 (8.11)  | 44.35 (3.51)  | 44.29 (1.15)  | 40.91 (1.29)  | 41.26 (0.82)  | 37.49 (0.46)  |
| WIDE  | Plain        | 25.28 (24.15) | 57.29 (n/a) | 48.39 (0.00) | 43.78 (0.50) | 33.86 (15.16) | 31.10 (17.07) | 45.08 (2.58)  | 46.79 (1.19)  | 46.25 (1.72)  | 36.42 (1.66)  |
|       | Dropout      | 30.76 (26.11) | 62.27 (n/a) | 55.96 (0.00) | 34.94 (0.81) | 43.74 (8.80)  | 33.94 (19.73) | 47.92 (1.37)  | 45.04 (6.85)  | 46.19 (5.31)  | 42.41 (-0.93) |
|       | Weight Decay | 22.78 (27.19) | 59.62 (n/a) | 47.77 (0.00) | 44.12 (0.06) | 42.44 (8.36)  | 37.45 (13.47) | 47.24 (1.60)  | 48.11 (0.62)  | 48.17 (0.82)  | 39.16 (1.41)  |
| DEEP  | Plain        | 20.82 (20.60) | 51.04 (n/a) | 34.75 (0.00) | 8.06 (3.21)  | 24.53 (12.15) | 29.14 (7.92)  | 33.58 (0.91)  | 32.28 (2.58)  | 27.78 (3.14)  | 27.51 (0.47)  |
|       | Dropout      | 23.05 (27.30) | 59.58 (n/a) | 46.22 (0.00) | 27.54 (2.79) | 32.76 (15.09) | 31.16 (17.06) | 39.07 (5.02)  | 37.89 (7.78)  | 36.85 (4.87)  | 33.64 (-0.61) |
|       | Weight Decay | 19.48 (21.27) | 54.60 (n/a) | 36.99 (0.00) | 30.25 (2.02) | 26.04 (9.51)  | 22.47 (13.19) | 19.35 (13.62) | 33.15 (1.16)  | 31.71 (1.39)  | 25.62 (-0.15) |

overfitting. Dropout leading to more forgetting is something we also observe for many other methods (see blue boxes), and exacerbates as the task sequence grows in length.

*Joint Training and PackNet.* Mainly relish higher accuracies with both regularization setups. By construction, they benefit from the regularization without interference issues. PackNet even reaches a top performance of almost 56 percent, that is 7 percent higher than closest competitor MAS.

*HAT* mainly favours weight decay of the model parameters, excluding embedding parameters. The embeddings are already prone to sparsity regularization, for which we found troublesome learning when included for weight decay as well. Dropout results in worse results, probably due to interfering with the unit-based masking. However, difficulties regarding asymmetric capacity in the DEEP model seem to be alleviated using either type of regularization.

*SI* most saliently thrives with regularization (with increases in performance of around 10 percent, see green box), which we previously found to be sensitive to overfitting. The regularization aims to find a solution to a well-posed problem, stabilizing the path in parameter space. Therefore, this might provide a better importance weight estimation along the path. Nonetheless, SI remains noncompetitive with leading average accuracies of other methods.

*EWC and MAS* suffer from interference with additional regularization. For dropout, more redundancy in the model means less unimportant parameters left for learning new tasks. For weight decay, parameters deemed important for previous tasks are also decayed in each iteration, affecting performance on older tasks. A similar effect was noticed in [10]. However, in some cases, the effect of this interference is again compensated for by having better initial models. EWC only benefits from dropout on the WIDE and DEEP models, and similar to MAS prefers no regularization on the SMALL model.

*LwF and EBLL* suffer from using dropout, with only the DEEP model significantly improving performance. For weight decay the methods follow the general trend, enhancing the WIDE net accuracies.

*IMM* with dropout exhibits higher performance only for the WIDE and DEEP model, coming closer to the performance obtained by the other methods.

*iCaRL and GEM.* Similar to the observations in Table 6, there is a striking increased forgetting for dropout for all methods in Table 7. Especially iCaRL shows increased average forgetting, albeit consistently accompanied with higher average accuracy. Except for the SMALL model, all models for

the replay baselines benefit from dropout. For GEM this benefit is only notable for the WIDE and DEEP models. Weight decay doesn't improve average accuracy for the replay methods, nor forgetting, except mainly for the WIDE model. In general, the influence of regularization seems limited for replay methods in Table 7 compared to non-replay based methods in Table 6.

## 6.5 Effects of a Real-World Setting

*iNaturalist and RecogSeq.* Up to this point all experiments conducted on Tiny Imagenet are nicely balanced, with an equal amount of data and classes per task. In further experiments we scrutinize highly unbalanced task sequences, both in terms of classes and available data per task.

We train AlexNet pretrained on Imagenet, and track task performance for RecogSeq and iNaturalist in Figs. 3 and 4a. The experiments exclude both noncompetitive HAT results due to problems of asymmetric capacity allocation discussed in Appendix B.5 available in the online supplemental material, and replay methods as they lack a policy to cope with unbalanced data, which would make comparison highly biased to our implementation.

### 6.5.1 Discussion

*iNaturalist.* Overall PackNet shows the highest average accuracy, tailgated by mode-IMM with superior forgetting by exhibiting positive backward transfer. Both PackNet and mode-IMM attain accuracies very close or even surpassing joint training, such as for Task 1 and Task 8.

*Performance Drop.* Evaluation on the first 4 tasks shows salient dips when learning Task 5 for prior-based methods

TABLE 7  
Replay methods: Dropout ( $p = 0.5$ ) and Weight Decay ( $\lambda = 10^{-4}$ ) Regularization for Tiny Imagenet Models

| Model |              | R-PM 4.5k     | R-FM 4.5k     | GEM 4.5k      | iCaRL 4.5k    |
|-------|--------------|---------------|---------------|---------------|---------------|
| SMALL | Plain        | 36.97 (12.21) | 39.60 (9.57)  | 39.44 (7.38)  | 43.22 (-1.39) |
|       | Dropout      | 35.50 (12.02) | 35.75 (9.87)  | 33.85 (6.25)  | 44.82 (4.83)  |
|       | Weight Decay | 35.57 (13.38) | 39.11 (9.49)  | 37.64 (5.11)  | 44.90 (-0.81) |
| BASE  | Plain        | 36.09 (10.96) | 37.31 (9.21)  | 45.13 (4.96)  | 47.27 (-1.11) |
|       | Dropout      | 43.32 (10.59) | 45.76 (7.38)  | 36.09 (12.13) | 48.42 (2.68)  |
|       | Weight Decay | 36.11 (10.13) | 37.78 (8.01)  | 38.05 (8.74)  | 45.97 (-2.32) |
| WIDE  | Plain        | 36.47 (12.45) | 39.25 (9.26)  | 40.32 (6.97)  | 44.20 (-1.43) |
|       | Dropout      | 42.20 (12.31) | 45.51 (8.85)  | 42.76 (6.33)  | 45.59 (2.74)  |
|       | Weight Decay | 39.75 (8.28)  | 38.73 (10.01) | 45.27 (5.92)  | 46.58 (-1.38) |
| DEEP  | Plain        | 27.61 (7.14)  | 32.26 (3.33)  | 29.66 (6.57)  | 36.12 (-0.93) |
|       | Dropout      | 34.42 (9.57)  | 37.22 (7.65)  | 32.75 (8.15)  | 41.77 (3.58)  |
|       | Weight Decay | 26.70 (9.01)  | 31.70 (4.86)  | 27.26 (5.94)  | 33.70 (-0.47) |

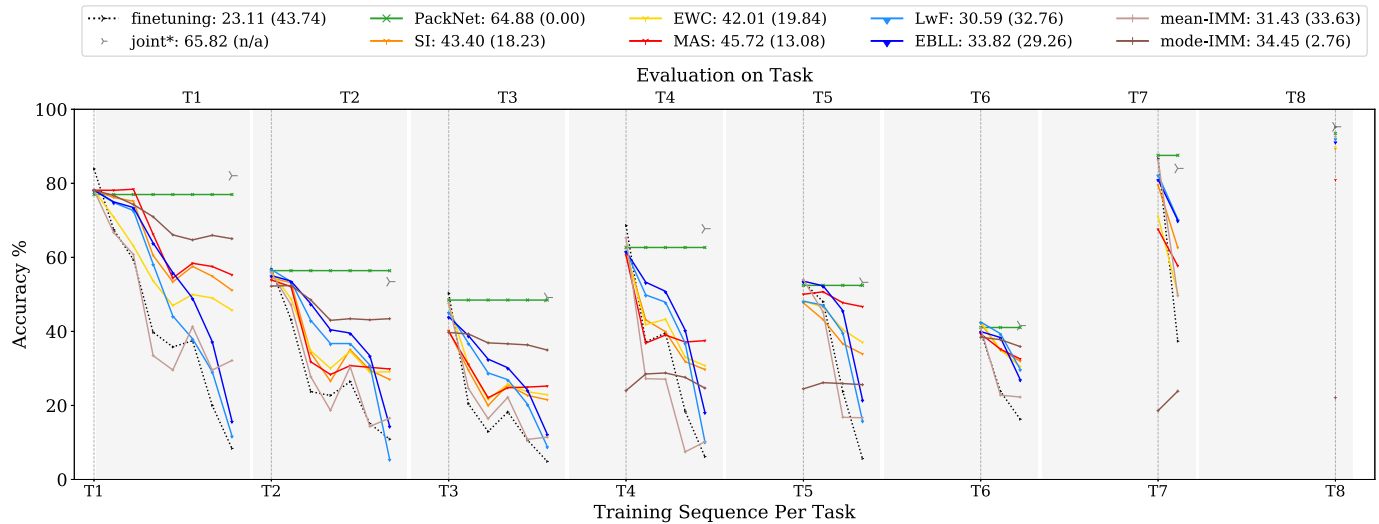


Fig. 3. RecogSeq dataset sequence results, reporting average accuracy (forgetting).

EWC, SI and MAS. In the random ordering Task 5 is an extremely easy task of supercategory ‘Fungi’ which contains only 5 classes and few data. Using expert gates to measure relatedness, the first four tasks show no particularly salient relatedness peaks or dips for Task 5. Instead, forgetting might rather be caused by the limited amount of training data, with only a few target classes, enforcing the network to overly fit to this task.

SI. For iNaturalist we did not observe overfitting, which might be the cause for stable SI behaviour in comparison to Tiny Imagenet.

LwF and EBLL. LwF catastrophically forgets on the unbalanced dataset with 13.77 percent forgetting, which is in high contrast with the results acquired on Tiny Imagenet. The supercategories in iNaturalist constitute a completely different task, imposing severe distribution shifts between the tasks. In the contrary, Tiny Imagenet is constructed from a subset of randomly collected classes, implying similar levels of homogeneity between task distributions. On top, EBLL constrains the new task features to reside closely to the optimal presentation for previous task features, which for the random ordering enforces forgetting to nearly halve from 13.77 to 7.51 percent, resulting in a striking 7.91 percent increase in average accuracy over LwF (from 45.39 to 53.30 percent).

RecogSeq shows similar findings to iNaturalist for LwF and EBLL, exacerbating in performance as they are subject to severe distribution shifts between tasks. This is most notable for the fierce forgetting when learning the last SVHN task. Besides the data-focused methods, we also observe deterioration for the prior-focused regularization-based methods. This shows increasing difficulty of these methods to cope with highly different recognition tasks. By contrast, PackNet thrives in this setup, taking the lead by high margin and approaching joint performance. In its advantage, PackNet freezes the previous task parameters, to impose zero forgetting regardless of the task dissimilarities. For all continual learning methods, Appendix B.4 available in the online supplemental material reports reproduced results similar to the original RecogSeq setup [5], [13], showcasing hyperparameter sensitivity and an urge for hyperparameter robustness.

## 6.6 Effects of Task Order

In this experiment we scrutinize how changing the task order affects both the balanced (Tiny Imagenet) and unbalanced (iNaturalist) task sequences. Our hypothesis resembles curriculum learning [95], implying knowledge is better captured starting with the general easier tasks followed by harder specific tasks. Instead, experimental results on both datasets exhibit ordering agnostic behaviour, corresponding to similar previous findings [96].

*Tiny Imagenet.* The previously discussed results of parameter isolation and regularization-based methods on top of Table 4, and replay methods on top of Table 5 concern a random ordering. We define a new order based on task difficulty, by measuring the accuracy over the 4 models obtained on held-out datasets for each of the tasks. This results in an ordering from *hard to easy* consisting of task sequence [5, 7, 10, 2, 9, 8, 6, 4, 3, 1] from the random ordering, with the inverse order equal to the *easy to hard* ordering.

*iNaturalist* constitutes three orderings defined as follows. First, we alphabetically order the supercategories to define the random ordering, as depicted in Fig. 4a. Further, the two other orderings start with task ‘Aves’ comprising most data, whereafter the remaining tasks are selected based on the average relatedness to the already included tasks in the sequence. Relatedness is measured using Expert Gate autoencoders following [5]. Selecting on highest relatedness results in the *related ordering* in Fig. 4b, whereas selecting on lowest relatedness ends up with the *unrelated ordering* in Fig. 4c. We refer to Table 8 for the configuration of the three different orderings.

### 6.6.1 Discussion

*Tiny Imagenet.* The main observations on the random ordering for the BASE model in Section 6.2 and model capacity in Section 6.3 remain valid for the two additional orderings, with PackNet and iCaRL competing for highest average accuracy, subsequently followed by MAS and LwF. In the following, we will instead focus on general observations between the three different orderings.

*Task Ordering Hypothesis.* Starting from our curriculum learning hypothesis we would expect the easy-to-hard

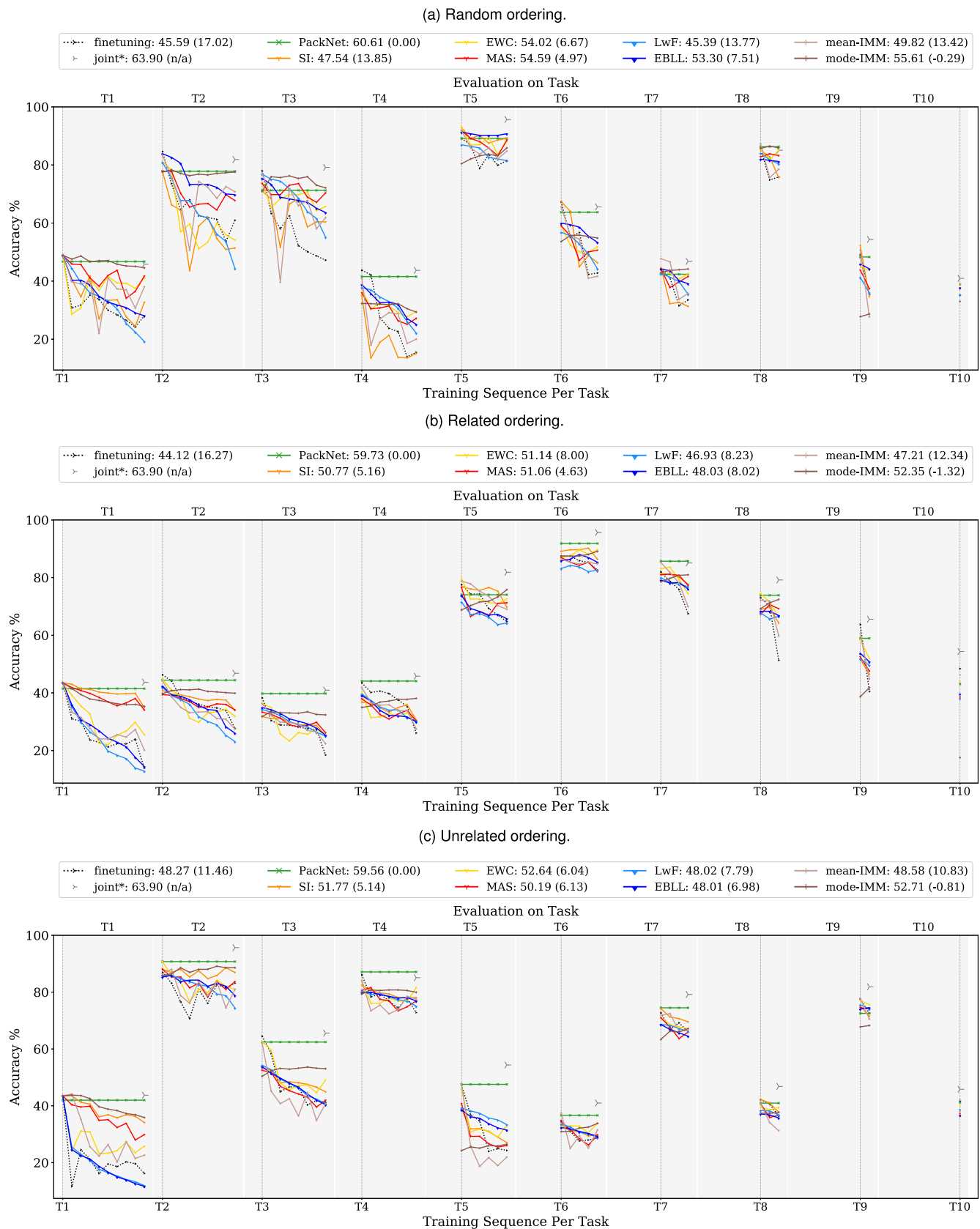


Fig. 4. Continual learning methods accuracy plots for three different orderings of the iNaturalist dataset.

ordering to enhance performance w.r.t. the random ordering, and the opposite effect for the hard-to-easy ordering. However, especially SI and EWC show unexpected better results

for the hard-to-easy ordering than for the easy-to-hard ordering. For PackNet and MAS, we see a systematic improvement when switching from hard-to-easy to easy-to-hard ordering.



TABLE 8  
The Unbalanced iNaturalist Task Sequence Details for Random, Related, and Unrelated Orderings

| Task      | Classes | Samples |      |       | Ordering |      |        |
|-----------|---------|---------|------|-------|----------|------|--------|
|           |         | Train   | Val  | Test  | Rand.    | Rel. | Unrel. |
| Amphibia  | 28      | 5319    | 755  | 1519  | 1        | 4    | 10     |
| Animalia  | 10      | 1388    | 196  | 397   | 2        | 5    | 9      |
| Arachnida | 9       | 1192    | 170  | 341   | 3        | 8    | 7      |
| Aves      | 314     | 65897   | 9362 | 18827 | 4        | 1    | 1      |
| Fungi     | 5       | 645     | 91   | 184   | 5        | 6    | 2      |
| Insecta   | 150     | 26059   | 3692 | 7443  | 6        | 9    | 3      |
| Mammalia  | 42      | 8646    | 1231 | 2469  | 7        | 2    | 8      |
| Mollusca  | 13      | 1706    | 241  | 489   | 8        | 7    | 4      |
| Plantae   | 237     | 35157   | 4998 | 10045 | 9        | 10   | 5      |
| Reptilia  | 56      | 10173   | 1447 | 2905  | 10       | 3    | 6      |

The gain is, however, relatively small. Overall, the impact of the task order seems insignificant.

**Replay Buffer.** Introducing the two other orderings, we now observe that iCaRL doesn't always improve when increasing the replay buffer size for the easy-to-hard ordering (see SMALL and WIDE model). More exemplars induce more samples to distill knowledge from previous tasks, but might deteriorate stochasticity in the estimated feature means in the nearest-neighbour classifier. GEM does not as consistently benefit from increased replay buffer size (GEM 9k), which could also root in the reduced stochasticity of the constraining gradients from previous tasks.

**iNaturalist.** The general observations for the random ordering remain consistent for the other orderings as well, with PackNet and mode-IMM the two highest performing methods.

**Surpassing Joint Training.** In all three orderings, PackNet consistently performs slightly better than Joint training for the *Mollusca* task. This is an easy task with only 13 classes to discriminate, with PackNet enabling beneficial forward transfer from previous tasks.

**Small Task Dip.** For the random ordering we observed a dip in accuracies for all preceding tasks when learning the very small *Fungi* task. Furthermore, for the unrelated ordering, the *Fungi* (Task 2) also experiences a performance dip for finetuning and EWC in Fig. 4b (see leftmost evaluation

panel of Task 1). On top, EWC shows the same behaviour for *Fungi* (Task 6) also in the related ordering (Fig. 4b), and therefore seems to be most susceptible to highly varying task constitutions, and especially smaller tasks.

**LwF and EBL.** Where EBL improves LwF with a significant 7.91 percent accuracy in the random ordering, and 1.1 percent in the related ordering, it performs similar to LwF for the unrelated ordering.

**SI and First Task.** As observed on Tiny Imagenet as well, SI is sensitive to overfitting when estimating importance weights through the path integral. The same behaviour can be observed for the random ordering which starts with a rather small task *Amphibia* (5k training samples) showing very unstable learning curves. The two other orderings start from task *Aves* (65k training samples) from which the amount of data might act as a regularizer and prevent overfitting. Hence, the path integral can be stabilized, resulting in very competitive results compared to EWC and MAS.

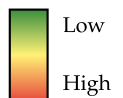
## 6.7 Qualitative Comparison

The previous experiments have a strong focus on coping with catastrophic forgetting, whereas by construction each method features additional advantages and limitations w.r.t. the other methods. In Table 9 we visualize relative GPU memory requirements and computation time for each of the methods in a heatmap, and formalize the extra required storage during the lifetime of the continual learner. Further, we emphasize task-agnosticity and privacy in our comparison. Note that we present the results in an indicative heatmap for a more high level relative comparison, as the results highly depend on our implementation. Details of the setup are provided in Appendix A available in the online supplemental material.

**GPU Memory** for training is slightly higher for EBL, which requires the autoencoder to be loaded in memory as well. At test time all memory consumption is more or less equal, but shows some deviations for GEM, HAT and PackNet due to our implementation. PackNet and HAT respectively load additional masks and embeddings in memory for evaluation. However, the BASE model size (54 MB), and hence additional mask or embedding size, are rather small compared to concurrently loaded data (200 images of size

TABLE 9  
Qualitative Comparison of the Compared Continual Learning Methods

| Category          | Method   | Memory |      | Compute |       | Task-agnostic possible | Privacy issues | Additional required storage       |
|-------------------|----------|--------|------|---------|-------|------------------------|----------------|-----------------------------------|
|                   |          | train  | test | train   | test  |                        |                |                                   |
| Replay-based      | iCaRL    | 1.24   | 1.00 | 5.63    | 45.61 | ✓                      | ✓              | $M + R$                           |
|                   | GEM      | 1.07   | 1.29 | 10.66   | 3.64  | ✓                      | ✓              | $\mathcal{T} \cdot M + R$         |
| Reg.-based        | LwF      | 1.07   | 1.10 | 1.29    | 1.86  | ✓                      | ✗              | $M$                               |
|                   | EBL      | 1.53   | 1.08 | 2.24    | 1.34  | ✓                      | ✗              | $M + \mathcal{T} \cdot A$         |
|                   | SI       | 1.09   | 1.05 | 1.13    | 1.61  | ✓                      | ✗              | $3 \cdot M$                       |
|                   | EWC      | 1.09   | 1.05 | 1.11    | 1.88  | ✓                      | ✗              | $2 \cdot M$                       |
|                   | MAS      | 1.09   | 1.05 | 1.16    | 1.88  | ✓                      | ✗              | $2 \cdot M$                       |
|                   | mean-IMM | 1.01   | 1.03 | 1.09    | 1.18  | ✓                      | ✗              | $\mathcal{T} \cdot M$             |
|                   | mode-IMM | 1.01   | 1.03 | 1.24    | 1.00  | ✓                      | ✗              | $2 \cdot \mathcal{T} \cdot M$     |
|                   | PackNet  | 1.00   | 1.94 | 2.66    | 2.40  | ✗                      | ✗              | $\mathcal{T} \cdot M[\text{bit}]$ |
| Param. iso.-based | HAT      | 1.21   | 1.17 | 1.00    | 2.06  | ✗                      | ✗              | $\mathcal{T} \cdot U$             |



The heatmap gives relative numbers to the minimal observed value in the column. Denoting number of seen tasks ( $\mathcal{T}$ ), replay buffer size ( $R$ ), autoencoder size ( $A$ ), size for model parameters ( $M$ ), with corresponding binary parameter mask  $M[\text{bit}]$ , and unit-based embeddings ( $U$ ).

Authorized licensed use limited to: University of Illinois at Chicago Library. Downloaded on March 19, 2025 at 10:29:19 UTC from IEEE Xplore. Restrictions apply.

TABLE 10  
Summary of Our Main Findings

| Method               | Best Avg. Acc. |             |          | Suggested Regularizer | Suggested Model | Comments                                                                                                                                                                                             |
|----------------------|----------------|-------------|----------|-----------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                      | Tiny Imagenet  | iNaturalist | RecogSeq |                       |                 |                                                                                                                                                                                                      |
| Replay               |                |             |          |                       |                 | <ul style="list-style-type: none"><li>- Least sensitive model capacity/regularization</li><li>- Privacy issues storing raw images</li><li>- No clear policy for unbalanced tasks</li></ul>           |
| iCaRL 4.5k (9k)      | 48.55 (49.94)  | x           | x        | dropout               | S/B/W           | <ul style="list-style-type: none"><li>- Lead performance</li><li>- Designed for class incremental setup</li><li>- Continual exemplar management</li><li>- Nearest-neighbor classifier</li></ul>      |
| GEM 4.5k (9k)        | 45.27 (44.23)  | x           | x        | none/dropout          | S/B/W           | <ul style="list-style-type: none"><li>- Designed for online continual setup</li><li>- Sensitive to amount of epochs</li></ul>                                                                        |
| Regularization-based |                |             |          |                       |                 |                                                                                                                                                                                                      |
| LwF                  | 48.11          | 48.02       | 30.59    | L2                    | W               | <ul style="list-style-type: none"><li>- Invigorated by WIDE model</li><li>- Requires sample outputs on previous model</li><li>- Forgetting buildup for older dissimilar tasks</li></ul>              |
| EBLL                 | 48.17          | 53.30       | 33.82    | L2                    | W               | <ul style="list-style-type: none"><li>- Margin over LwF</li><li>- Autoencoder gridsearch for unbalanced tasks</li></ul>                                                                              |
| SI                   | 43.74          | 51.77       | 43.40    | dropout/L2            | B/W             | <ul style="list-style-type: none"><li>- Efficient training time over EWC/MAS</li><li>- Requires dropout or L2 (prone to overfitting)</li><li>- Most affected by task ordering</li></ul>              |
| EWC                  | 45.13          | 54.02       | 42.01    | none                  | S               | <ul style="list-style-type: none"><li>- Invigorated by SMALL capacity model</li><li>- Deteriorates on WIDE model</li></ul>                                                                           |
| MAS                  | 48.98          | 54.59       | 45.72    | none                  | B/W             | <ul style="list-style-type: none"><li>- Lead regularization-based performance</li><li>- Hyperparameter robustness</li><li>- Unsupervised importance weight estimation</li></ul>                      |
| mean-IMM             | 32.42          | 49.82       | 31.43    | none                  | B/W             | <ul style="list-style-type: none"><li>- mode-IMM outperforms mean-IMM</li><li>- Both require additional merging step</li></ul>                                                                       |
| mode-IMM             | 42.41          | 55.61       | 34.45    | none                  | B/W             |                                                                                                                                                                                                      |
| Parameter isolation  |                |             |          |                       |                 | <ul style="list-style-type: none"><li>- Efficient memory</li><li>- Prevents scalable class incremental setup</li></ul>                                                                               |
| PackNet              | 55.96          | 60.61       | 64.88    | dropout/L2            | S/B/W           | <ul style="list-style-type: none"><li>- Lead performance</li><li>- No forgetting (after compression)</li><li>- Requires retraining after compression</li><li>- Heuristic parameter pruning</li></ul> |
| HAT                  | 44.19          | x           | x        | L2                    | B/W             | <ul style="list-style-type: none"><li>- Nearly no forgetting (nearly binary masks)</li><li>- End-to-end attention mask learning</li><li>- Saturating low-level feature capacity</li></ul>            |

We report best results over all experiments, i.e., including regularization experiments for Tiny Imagenet. The SMALL, BASE, WIDE models are denoted as S,B,W, and weight decay as L2.

$64 \times 64 \times 3$ ). Double test time memory for PackNet is due to our implementation loading all masks at once for faster prediction, with the cost of increased memory consumption of the masks (factor 10 increase, one mask per task).

Computation time for training is doubled for EBLL and PackNet (see light green), as they both require an additional training step, respectively for the autoencoder and compression phase. iCaRL training requires additional forward passes for exemplar sets (factor 5 increase). On top of that, GEM requires backward passes to acquire exemplar set gradients, and solves a quadratic optimization problem (factor 10 increase). During testing, the iCaRL nearest-neighbor classifier is vastly outperformed by a regular softmax classifier in terms of computation time (factor 45 increase).

Storage indicates the additional memory required for the continual learning method. LwF and iCaRL both store the previous task model for knowledge distillation (although one could also store the recorded outputs instead). On top of that, EBLL stores previous task autoencoders, small in size  $A$  relative to  $M$ . Further, iCaRL and GEM store exemplars, with GEM additional exemplar gradients. Prior-focused methods EWC and MAS store importance weights

and the previous task model, with SI also requiring a running estimate of the current task importance weights. IMM, when naively implemented, stores for each task the model after training, with mode-IMM additionally storing importance weights. More storage efficient implementations propagate a merged model instead. Finally, PackNet requires task-specific masks equal to the amount parameters in the model ( $M[\text{bit}]$ ), and HAT obtains unit-based embeddings.

*Task-agnostic and privacy.* At deployment, PackNet and HAT require the task identifier for a given sample to load the appropriate masks and embeddings. Therefore, this setup inherently prevents task-agnostic inference with a single shared head between all tasks. Next, we emphasize the privacy issues for replay-based methods iCaRL and GEM, storing raw images as exemplars.

## 6.8 Experiments Summary

We summarize our main findings for each method in Table 10. Overall, PackNet seems the best performer by high margin on all three datasets, tailored for the task incremental multi-head setup. However, it refrains applicability in class incremental setups and is confined to the initial model capacity, as

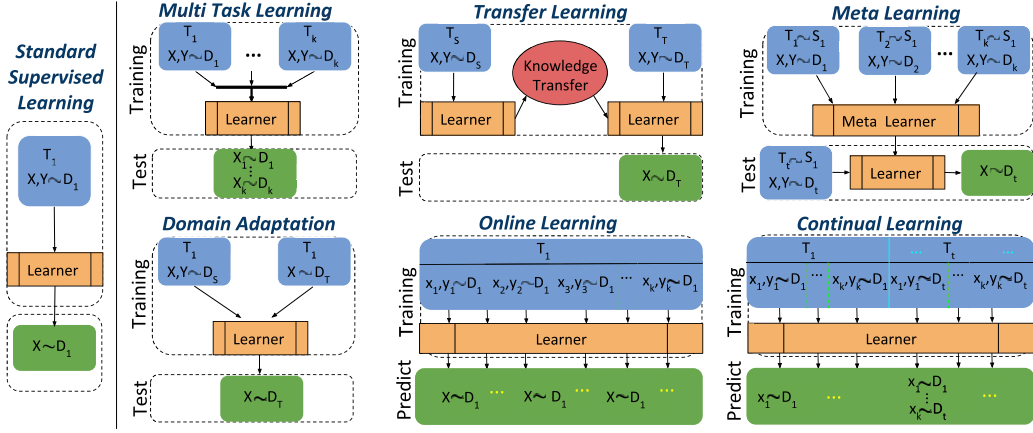


Fig. 5. Main setup of related machine learning fields, illustrating the differences with general continual learning settings.

exemplified in supplemental experiments in Appendix B.6 available online. In the regularization-based methods, MAS exhibits hyperparameter robustness and consistent lead performance over the datasets. Competing mode-IMM thrives in the iNaturalist setup, but lacks robustness to the other setups. Further, LwF and EBLI show competitive performance for relatively similar tasks, but collapse for the significant distribution shifts in RecogSeq. Replay-methods iCarl and GEM are not designed for a task incremental setup, but nonetheless show competitive performance with iCarl improving on larger memory size.

Weight decay and dropout show consistent merits only for specific methods and increased model capacity in DEEP models exhibits inferior performance in the scope of our experiments. Finally, presenting the tasks in different orderings to the continual learner has insignificant performance effects.

## 7 LOOKING AHEAD

In the experiments performed in this survey, we have considered an task incremental learning setting. In this setting, tasks are received one at the time and offline training is performed on their associated training data until convergence. Knowledge of the task boundaries is required (i.e., when tasks switch) and allows for multiple passes over large batches of training data. Hence, it resembles a relaxation of the desired continual learning system that is more likely to be encountered in practice. Below, we describe the general setting in which continual learning methods are expected to be deployed, and outline the main characteristics to be considered by upcoming methods. *The General Continual Learning Setting* considers an infinite stream of training data where at each time step, the system receives a (number of) new sample(s) drawn non i.i.d from a current distribution  $D_t$  that could itself experience sudden or gradual changes. The main goal is to optimally learn a function that minimizes a predefined loss on the new sample(s) without interfering with, and possibly improving on, those that were learned previously. Desiderata of an ideal continual learning scheme include:

1. *Constant Memory.* To avoid unbounded systems, the consumed memory should be constant w.r.t. the number of tasks or length of the data stream.

2. *No Task Boundaries.* Learning from the input data without requiring clear task divisions makes continual learning applicable to any never-ending data stream.
3. *Online Learning* Without demanding offline training of large batches or separate tasks introduces fast acquisition of new information.
4. *Forward Transfer* Or zero-shot learning indicates the importance of previously acquired knowledge to aid the learning of new tasks by increased data efficiency.
5. *Backward Transfer* Aims at retaining previous knowledge and preferably improving it when learning future related tasks.
6. *Problem Agnostic* Continual learning is not limited to a specific setting (e.g., only classification).
7. *Adaptive Systems* learn from available unlabeled data as well, opening doors for adaptation to specific user data.
8. *No Test Time Oracle* Providing the task label should be required for prediction.
9. *Task Revisiting* of previously seen tasks should enable enhancement of the corresponding task knowledge.
10. *Graceful Forgetting.* Given an unbounded system and infinite stream of data, selective forgetting of trivial information is an important mechanism to achieve balance between stability and plasticity.

After establishing the general continual learning setting and the main desiderata, it is important to identify the critical differences between continual learning and other closely related machine learning fields that share some of the continual learning characteristics mentioned above.

## 8 RELATED MACHINE LEARNING FIELDS

The ideas depicted in the continual learning desiderata of knowledge sharing, adaptation and transfer have been studied previously in machine learning with development in related fields. We will describe each of them briefly and highlight the main differences with continual learning (see also Fig. 5).

*Multi-Task Learning* (MTL) considers learning multiple related tasks simultaneously using a set or subset of shared parameters. It aims for a better generalization and a reduced overfitting using shared knowledge extracted from the related tasks. We refer to [19] for a survey on the topic. As



in our experiments, an MTL baseline is represented by a *joint* learning of all tasks simultaneously. There is no adaptation after the model has been deployed, as opposed to continual learning.

*Transfer Learning* aims to aid the learning process of a given task (the target) by exploiting knowledge acquired from another task or domain (the source). Transfer learning is mainly concerned with the forward transfer desiderata of continual learning. However, it does not involve any continuous adaptation after learning the target task. Moreover, performance on the source task(s) is not taken into account during transfer learning. We refer to [97] for a survey.

*Domain Adaptation* is a subfield of transfer learning with the same source and target tasks, but drawn from different input domains. The goal is to adapt a model trained on the source domain to perform well on the target domain, which has only unlabelled data or has only few labels). In other words, it relaxes the classical machine learning assumption of having training and test data drawn from the same distribution [98]. As mentioned above for transfer learning, domain adaptation is unidirectional and does not involve any accumulation of knowledge [4].

*Learning to Learn (Meta Learning)*. The old definition of learning to learn was referring to the concept of improving the learning behaviour of a model with training experience. However, more recently, the common interpretation is the ability for a faster adaptation on a task with few examples given a large number of training tasks. While these ideas seem quite close to continual learning, meta learning still follows the same offline training assumption, but with data randomly drawn from a task training distribution, and test data being tasks with few examples. Hence, it is not capable, alone, of preventing forgetting on those previous tasks.

*Online Learning*. In traditional offline learning, the entire training data has to be made available prior to learning the task. On the contrary, online learning studies algorithms that learn to optimize predictive models over a stream of data instances sequentially. We refer to [99], [100] for surveys on the topic. Note that online learning assumes an i.i.d data sampling procedure and considers a single task domain, which sets it apart from continual learning.

*Open World Learning* [101], [102] deals with the problem of detecting new classes at test time, hence avoiding wrong assignments to known classes. When those new classes are then integrated in the model, it meets the problem of incremental learning. As such, open world learning can be seen as a subtask of continual learning.

## 9 CONCLUSION

In this work, we scrutinized recent state-of-the-art continual learning methods, confining the study to task incremental classification with a multi-head setup, as mainly applied in literature. Within these outlines we proposed a constructive taxonomy and tested each of the compared methods in an attempt to answer several critical questions in the field. In order to fulfill the urge for a fair comparison compliant with the continual learning paradigm, we proposed a novel continual hyperparameter framework which dynamically defines forgetting related hyperparameters, i.e., the stability-plasticity trade-off is determined in a continual fashion with

only the current task data available. The overview of our experimental findings in Table 10 shed an empirical light on method strengths and weaknesses, supplemented by recommendations for model capacity and the use of dropout or weight decay. The experiments extend to three datasets, namely Tiny Imagenet, iNaturalist and RecogSeq, from which the latter two resemble real-world data sequences to challenge methods with an unbalanced data distribution and highly varying tasks. On top, we addressed the influence of task ordering and found it minimally influential towards the general trends of the compared continual learning methods.

Although the state-of-the-art made some significant progress to tackle catastrophic forgetting in neural networks, the majority of results originates from a highly confined setup, leaving numerous opportunities for further research to reach beyond classification, multi-head evaluation and a task incremental setup.

## ACKNOWLEDGMENTS

The authors would like to thank Huawei for funding this research. The work of Marc Masana was supported by the Generalitat de Catalunya under Grant 2019-FI\_B2-00189. The work of Rahaf Aljundi was supported by the FWO Scholarship.

## REFERENCES

- [1] D. Silver *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [2] O. Russakovsky *et al.*, "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, no. 3, pp. 211–252, 2015.
- [3] R. M. French, "Catastrophic forgetting in connectionist networks," *Trends Cogn. Sci.*, vol. 3, no. 4, pp. 128–135, 1999.
- [4] Z. Chen and B. Liu, "Lifelong machine learning," *Synthesis Lectures Artif. Intell. Mach. Learn.*, vol. 12, no. 3, pp. 1–207, 2018.
- [5] R. Aljundi, P. Chakravarty, and T. Tuytelaars, "Expert gate: Lifelong learning with a network of experts," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 3366–3375.
- [6] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient lifelong learning with A-GEM," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–20.
- [7] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *Neural Netw.*, vol. 113, pp. 54–71, 2019.
- [8] D. L. Silver and R. E. Mercer, "The task rehearsal method of lifelong learning: Overcoming impoverished data," in *Proc. Conf. Can. Soc. Comput. Studies Intell.*, 2002, pp. 90–101.
- [9] A. Rannen, R. Aljundi, M. B. Blaschko, and T. Tuytelaars, "Encoder based lifelong learning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1320–1328.
- [10] R. Aljundi, M. Rohrbach, and T. Tuytelaars, "Selfless sequential learning," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=Bkxbrn0cYX>
- [11] M. McCloskey and N. J. Cohen, "Catastrophic interference in connectionist networks: The sequential learning problem," *Psychol. Learn. Motivation*, 1989, vol. 24, pp. 109–165.
- [12] H. Shin, J. K. Lee, J. Kim, and J. Kim, "Continual learning with deep generative replay," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 2990–2999.
- [13] R. Aljundi, F. Babiloni, M. Elhoseiny, M. Rohrbach, and T. Tuytelaars, "Memory aware synapses: Learning what (not) to forget," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 139–154.
- [14] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. Torr, "Riemannian walk for incremental learning: Understanding forgetting and intransigence," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 532–547.
- [15] A. Gepperth and C. Karaoguz, "A bio-inspired incremental learning architecture for applied perceptual problems," *Cogn. Comput.*, vol. 8, no. 5, pp. 924–934, 2016.

- [16] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert, "iCaRL: Incremental classifier and representation learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 2001–2010.
- [17] A. Rosenfeld and J. K. Tsotsos, "Incremental learning through deep adaptation," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 3, pp. 651–663, Mar. 2020.
- [18] K. Shmelkov, C. Schmid, and K. Alahari, "Incremental learning of object detectors without catastrophic forgetting," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 3400–3409.
- [19] Y. Zhang and Q. Yang, "A survey on multi-task learning," 2017, *arXiv:1707.08114*.
- [20] S. Grossberg, *Studies of Mind and Brain : Neural Principles of Learning, Perception, Development, Cognition, and Motor Control*. Dordrecht, Netherlands: Reidel, 1982.
- [21] M. D. Lange, X. Jia, S. Parisot, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "Unsupervised model personalization while preserving privacy and scalability: An open problem," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 14 463–14 472.
- [22] C. S. Lee and A. Y. Lee, "Clinical applications of continual learning machine learning," *Lancet Digit. Health*, vol. 2, no. 6, pp. e279–e281, 2020.
- [23] P. Gupta, Y. Chaudhary, T. Runkler, and H. Schuetz, "Neural topic modeling with continual lifelong learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3907–3917.
- [24] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, "Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges," *Inf. Fusion*, vol. 58, pp. 52–68, 2020.
- [25] B. Pfülb and A. Gepperth, "A comprehensive, application-oriented study of catastrophic forgetting in DNNs," in *Proc. Int. Conf. Learn. Representations*, 2019. [Online]. Available: <https://openreview.net/forum?id=BkloRs0qK7>
- [26] S. Farquhar and Y. Gal, "Towards robust evaluations of continual learning," 2019, *arXiv:1805.09733*.
- [27] J. Kirkpatrick *et al.*, "Overcoming catastrophic forgetting in neural networks," *Proc. Nat. Acad. Sci.*, vol. 114, no. 13, pp. 3521–3526, 2017. [Online]. Available: <https://www.pnas.org/content/114/13/3521>
- [28] S.-W. Lee, J.-H. Kim, J. Jun, J.-W. Ha, and B.-T. Zhang, "Overcoming catastrophic forgetting by incremental moment matching," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 4652–4662.
- [29] R. Kemker, M. McClure, A. Abitino, T. L. Hayes, and C. Kanan, "Measuring catastrophic forgetting in neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3390–3398.
- [30] C. Fernando *et al.*, "PathNet: Evolution channels gradient descent in super neural networks," 2017, *arXiv:1701.08734*.
- [31] I. J. Goodfellow, M. Mirza, D. Xiao, A. Courville, and Y. Bengio, "An empirical investigation of catastrophic forgetting in gradient-based neural networks," 2015, *arXiv:1312.6211*.
- [32] Y.-C. Hsu, Y.-C. Liu, A. Ramasamy, and Z. Kira, "Re-evaluating continual learning scenarios: A categorization and case for strong baselines," 2018, *arXiv:1810.12488*.
- [33] M. D. Lange and T. Tuytelaars, "Continual prototype evolution: Learning online from non-stationary data streams," in *Proc. Int. Conf. Learn. Representations*, 2020, pp. 1–18.
- [34] R. M. French, "Semi-distributed representations and catastrophic forgetting in connectionist networks," *Connection Sci.*, vol. 4, no. 3–4, pp. 365–377, 1992.
- [35] R. M. French, "Dynamically constraining connectionist networks to produce distributed, orthogonal representations to reduce catastrophic interference," *Network*, vol. 1111, 1994, Art. no. 1.
- [36] J. K. Kruschke, "Alcove: An exemplar-based connectionist model of category learning," *Psychol. Rev.*, vol. 99, no. 1, 1992, Art. no. 22.
- [37] J. K. Kruschke, "Human category learning: Implications for back-propagation models," *Connection Sci.*, vol. 5, no. 1, pp. 3–36, 1993.
- [38] S. A. Sloman and D. E. Rumelhart, "Reducing interference in distributed memories through episodic gating," *Essays in Honor of WK Estes*. Mahwah, NJ, USA: Lawrence Erlbaum Associates, Inc., 1992, vol. 1, pp. 227–248.
- [39] A. Robins, "Catastrophic forgetting, rehearsal and pseudo-rehearsal," *Connection Sci.*, vol. 7, no. 2, pp. 123–146, 1995.
- [40] R. M. French, "Pseudo-recurrent connectionist networks: An approach to the 'sensitivity-stability' dilemma," *Connection Sci.*, vol. 9, no. 4, pp. 353–380, 1997.
- [41] J. Rueckl, "Jumpnet: A multiple-memory connectionist architecture," *Cogn Sci.*, no. 24, pp. 866–871, 1993.
- [42] B. Ans and S. Rousset, "Avoiding catastrophic forgetting by coupling two reverberating neural networks," *Comptes Rendus de l'Académie des Sciences-Series III-Sciences de la Vie*, vol. 320, no. 12, pp. 989–997, 1997.
- [43] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [44] A. Pentina and C. H. Lampert, "A pac-bayesian bound for life-long learning," in *ICML*, 2014, pp. 991–999.
- [45] P. Alquier *et al.*, "Regret bounds for lifelong learning," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2017, pp. 261–269.
- [46] A. Pentina and C. H. Lampert, "Lifelong learning with non-i.i.d. tasks," in *Proc. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1540–1548.
- [47] M.-F. Balcan, A. Blum, and S. Vempala, "Efficient representations for lifelong learning and autoencoding," in *Proc. Conf. Learn. Theory*, 2015, pp. 191–210.
- [48] R. Aljundi, M. Lin, B. Goujaud, and Y. Bengio, "Gradient based sample selection for online continual learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 11816–11825.
- [49] D. Rolnick, A. Ahuja, J. Schwarz, T. Lillicrap, and G. Wayne, "Experience replay for continual learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 350–360.
- [50] D. Isele and A. Cosgun, "Selective experience replay for lifelong learning," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 3302–3309.
- [51] A. Chaudhry *et al.*, "Continual learning with tiny episodic memories," 2019, *arXiv:1902.10486*.
- [52] C. Atkinson, B. McCane, L. Szymanski, and A. Robins, "Pseudo-recursion: Solving the catastrophic forgetting problem in deep neural networks," 2018, *arXiv:1802.03875*.
- [53] F. Lavda, J. Ramapuram, M. Gregorova, and A. Kalousis, "Continual classification learning using generative models," 2018, *arXiv:1810.10612*.
- [54] J. Ramapuram, M. Gregorova, and A. Kalousis, "Lifelong generative modeling," *Neurocomputing*, vol. 404, pp. 381–400, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0925231220303623>
- [55] D. Lopez-Paz and M. Ranzato, "Gradient episodic memory for continual learning," in *Proc. Int. Cong. Neural Inf. Process. Syst.*, 2017, pp. 6467–6476.
- [56] F. Zenke, B. Poole, and S. Ganguli, "Continual learning through synaptic intelligence," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 3987–3995.
- [57] X. Liu, M. Masana, L. Herranz, J. Van de Weijer, A. M. Lopez, and A. D. Bagdanov, "Rotate your networks: Better weight consolidation and less catastrophic forgetting," in *Proc. IEEE 24th Int. Conf. Pattern Recognit.*, 2018, pp. 2262–2268.
- [58] Z. Li and D. Hoiem, "Learning without forgetting," in *Pro. Eur. Conf. Comput. Vis.*, 2016, pp. 614–629.
- [59] H. Jung, J. Ju, M. Jung, and J. Kim, "Less-forgetting learning in deep neural networks," 2016, *arXiv:1607.00122*.
- [60] J. Zhang *et al.*, "Class-incremental learning via deep model consolidation," in *Proc. IEEE Winter Conf. Appl. Comput. Vis.*, 2020, pp. 1131–1140.
- [61] A. Mallya and S. Lazebnik, "PackNet: Adding multiple tasks to a single network by iterative pruning," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7765–7773.
- [62] A. Mallya, D. Davis, and S. Lazebnik, "Piggyback: Adapting a single network to multiple tasks by learning to mask weights," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 67–82.
- [63] J. Serrà, D. Surís, M. Miron, and A. Karatzoglou, "Overcoming catastrophic forgetting with hard attention to the task," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 4548–4557.
- [64] A. A. Rusu *et al.*, "Progressive neural networks," 2016, *arXiv:1606.04671*.
- [65] J. Xu and Z. Zhu, "Reinforced continual learning," in *Proc. Conf. Neural Inf. Process. Syst.*, 2018, pp. 899–908. [Online]. Available: <http://papers.nips.cc/paper/7369-reinforced-continual-learning.pdf>
- [66] M. Masana, X. Liu, B. Twardowski, M. Menta, A. D. Bagdanov, and J. van de Weijer, "Class-incremental learning: survey and performance evaluation," 2020, *arXiv:2010.15277*.
- [67] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. Conf. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
- [68] Y. Bengio and Y. LeCun, Eds., *Proc. Int. Conf. Learn. Representations*, 2014. [Online]. Available: <https://openreview.net/group?id=ICLR.cc/2014>

- [69] C. V. Nguyen, Y. Li, T. D. Bui, and R. E. Turner, "Variational continual learning," in *Proc. Int. Conf. Learn. Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=BkQqq0gRb>
- [70] H. Ahn, S. Cha, D. Lee, and T. Moon, "Uncertainty-based continual learning with adaptive regularization," in *Proc. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4394–4404.
- [71] C. Zeno, I. Golan, E. Hoffer, and D. Soudry, "Task agnostic continual learning using online variational bayes," 2019, *arXiv:1803.10123*.
- [72] R. Aljundi, K. Kelchtermans, and T. Tuytelaars, "Task-free continual learning," in *Proc. Int. Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11 254–11 263.
- [73] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *Stat*, vol. 1050, no. 9, 2015.
- [74] D. J. MacKay, "A practical bayesian framework for backpropagation networks," *Neural Comput.*, vol. 4, no. 3, pp. 448–472, 1992.
- [75] R. Pascanu and Y. Bengio, "Revisiting natural gradient for deep networks," 2013, *arXiv:1301.3584*.
- [76] J. Martens, "New insights and perspectives on the natural gradient method," *J. Mach. Learn. Res.*, vol. 21, pp. 1–76, 2020.
- [77] F. Huszár, "Note on the quadratic penalties in elastic weight consolidation," *Proc. Nat. Acad. Sci.*, vol. 115, no. 11, pp. E2496–E2497, 2018. [Online]. Available: <https://www.pnas.org/content/115/11/E2496>
- [78] J. Schwarz, W. Czarnecki, J. Luketina, A. Grabska-Barwinska, Y. W. Teh, R. Pascanu, and R. Hadsell, "Progress & compress: A scalable framework for continual learning," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4528–4537.
- [79] I. J. Goodfellow, O. Vinyals, and A. M. Saxe, "Qualitatively characterizing neural network optimization problems," 2015, *arXiv:1412.6544*.
- [80] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov, "Improving neural networks by preventing co-adaptation of feature detectors," 2012, *arXiv:1207.0580*.
- [81] M. Masana, T. Tuytelaars, and J. van de Weijer, "Ternary feature masks: Continual learning without any forgetting," 2020, *arXiv:2001.08714*.
- [82] Stanford, Tiny ImageNet Challenge, CS231N Course. Accessed: Sep. 2018. [Online]. Available: <https://www.kaggle.com/c/tiny-imagenet>
- [83] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 248–255.
- [84] iNaturalist dataset: FGVC5 workshop at CVPR 2018. 2018. [Online]. Available: <https://www.kaggle.com/c/inaturalist-2018>
- [85] M. Everingham, S. A. Eslami, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes challenge: A retrospective," *Int. J. Comput. Vis.*, vol. 111, no. 1, pp. 98–136, 2015.
- [86] M.-E. Nilsback and A. Zisserman, "Automated flower classification over a large number of classes," in *Proc. 6th IEEE Indian Conf. Comput. Vis. Graph. Image Process.*, 2008, pp. 722–729.
- [87] A. Quattoni and A. Torralba, "Recognizing indoor scenes," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2009, pp. 413–420.
- [88] P. Welinder, S. Branson, T. Mita, C. Wah, F. Schroff, S. Belongie, and P. Perona, "Caltech-UCSD birds 200," California Institute of Technology, 2010. [Online]. Available: <http://www.vision.caltech.edu/visipedia/CUB-200.html>
- [89] J. Krause, M. Stark, J. Deng, and L. Fei-Fei, "3D object representations for fine-grained categorization," in *Proc. IEEE Int. Conf. Comput. Vis. Workshops*, 2013, pp. 554–561.
- [90] S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, "Fine-grained visual classification of aircraft," 2013, *arXiv:1306.5151*.
- [91] T. E. De Campos, B. R. Babu, and M. Varma, "Character recognition in natural images," *Proc. 4th Int. Conf. Comput. Vis. Theory Appl.*, 2009, pp. 273–280.
- [92] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *Proc. Adv. Conf. Neural Inf. Process. Syst.*, 2011. [Online]. Available: [http://ufldl.stanford.edu/housenumbers/nips2011\\_housenumbers.pdf](http://ufldl.stanford.edu/housenumbers/nips2011_housenumbers.pdf)
- [93] C. Chung *et al.*, "Implementation of an integrated computerized prescriber order-entry system for chemotherapy in a multisite safety-net health system," *Amer. J. Health Syst. Pharm.*, vol. 752018, Art. no. 504.
- [94] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," in *Proc. Conf. Neural Inf. Process. Syst.*, 2012, pp. 1097–1105.
- [95] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. Int. Conf. Mach. Learn.*, 2009, pp. 41–48.
- [96] C. V. Nguyen, A. Achille, M. Lam, T. Hassner, V. Mahadevan, and S. Soatto, "Toward understanding catastrophic forgetting in continual learning," 2019, *arXiv:1908.01091*.
- [97] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.
- [98] G. Csurka, *Domain Adaptation in Computer Vision Applications*. Berlin, Germany: Springer, 2017.
- [99] S. Shalev-Shwartz *et al.*, "Online learning and online convex optimization," *Foundations Trends Mach. Learn.*, vol. 4, no. 2, pp. 107–194, 2012.
- [100] L. Bottou, "Online learning and stochastic approximations," *On-Line Learn. Neural Netw.*, vol. 17, no. 9, 1998, Art. no. 142.
- [101] H. Xu, B. Liu, L. Shu, and P. S. Yu, "Open-world learning and application to product classification," in *Proc. World Wide Web Conf.*, 2019, pp. 3413–3419. [Online]. Available: <https://doi.org/10.1145/3308558.3313644>
- [102] A. Bendale and T. Boulton, "Towards open world recognition," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 1893–1902.

**Matthias De Lange** received the MSc degree and laureate from KU Leuven, in Belgium 2018. He is currently working toward PhD degree from the Center for Processing Speech and Images of KU Leuven. His research interests encompass continual learning, computer vision, and user adaptation.

**Rahaf Aljundi** received the master's degree from Jean Monnet University, France, and University of Alicante, Spain, and the PhD degree from KU Leuven, Belgium, in 2019. She is currently a machine learning and vision researcher at Toyota Motor Europe. Her main research interests include continual learning, transfer learning, and active learning.

**Marc Masana** received the PhD degree from the Universitat Autònoma de Barcelona. He is currently a mathematician and computer science engineer working as a researcher with the Computer Vision Center. His research interests concern machine learning, network compression, and continual learning.

**Sarah Parisot** received the PhD degree from Ecole Centrale INRIA, Paris. She is currently a senior research scientist at Huawei Noah's Ark Lab. She held positions in both academia and industry. Her research interests include few-shot/unsupervised learning, computer vision and graphs.

**Xu Jia** received the MSc degree from the Dalian University of Technology, China, in 2013, and the PhD degree from KU Leuven, Belgium, in 2018. He is currently a senior researcher with Huawei Noah's Ark Lab. His research interests include image processing, computer vision, and machine learning.

**Aleš Leonardis** is currently a senior research scientist and computer vision team leader at a Huawei Technologies Noah's Ark Lab, London, UK. He is also the chair of robotics at the School of Computer Science, University of Birmingham, and a professor of computer and information science at the University of Ljubljana.

**Gregory Slabaugh** received the PhD degree from the Georgia Institute of Technology. He is currently a chief scientist of computer vision (Europe) with Huawei Noah's Ark Lab. He has worked in academia and industry. His research interests include computational photography and computer vision.

**Tinne Tuytelaars** is currently a full professor at the Center for Processing Speech and Images of KU Leuven. Her research interests encompass continual learning, robust image and video representations, and multi-modal analysis.

► For more information on this or any other computing topic, please visit our Digital Library at [www.computer.org/csdl](http://www.computer.org/csdl).